Master of Science in Engineering: Software Engineering
October 2026



A Performance-Driven Recommendation Framework for Systematic Database Selection in Mixed Traditional and Vector Data Systems

Leo Wedebrand
Simon Wargh

Faculty of Software Engineering, Blekinge Institute of Technology, 371 79 Karlskrona, Sweden

This thesis is submitted to the Faculty of Software Engineering at Blekinge Institute of Technology in partial fulfillment of the requirements for the degree of Master of Science in Engineering: Software Engineering. The thesis is equivalent to 20 weeks of full-time studies.

The authors declare that they are the sole authors of this thesis and that they have not used any sources other than those listed in the bibliography and identified as references. They further declare that they have not submitted this thesis to any other institution to obtain a degree.

AI tools were utilized in the writing of this thesis in accordance with BTH's AI Policy for TE2502; see the attached Declaration of AI Use for full details.

Contact Information:
Author(s):
Leo Wedebrand
E-mail: leae19@student.bth.se

Simon Wargh
E-mail: siag19@student.bth.se

University advisor:
Ricardo Britto, Ph. D.
Department of Software Engineering

Faculty of Software Engineering
Blekinge Institute of Technology
SE–371 79 Karlskrona, Sweden

Internet : www.bth.se
Phone : +46 455 38 50 00
Fax : +46 455 38 50 57

Abstract

Background. Selecting a database management system (DBMS) is a complex task, often requiring years of experience and an intricate understanding of the different available solutions. With further advancements in the field, the issue compounds, and the advent of vector data is no different.
Objectives In order to reach a possible solution to this problem, top-performing distributed solutions for a number of limited use cases were identified. Using this selection, a recommendation framework was developed. This framework was then evaluated in terms of correctness and usefulness.
Method. This thesis utilized the Design Science Research Process (DSRP) to guide the overall structure of the work. An initial literature review was conducted to identify the problem of selecting a database solution, and a second literature review was used to support the design of a DBMS selection framework. This framework was then evaluated using benchmark data obtained through a quasi-experiment, and insights about the usability of the framework were gained from static validation via an expert survey.
Results. A recommendation method in the form of a flow-chart was produced. The quasi-experiment partly reinforced assumptions about the impact of differing vector data quantity ratios, and challenged expectations regarding the capability from standalone solutions. The survey responses were generally favorable to this approach, with a majority of respondents noting the ease of use, though it was made clear the format had considerable limitations.
Conclusions. The introduction of vector data to the database selection process spawns additional considerations, especially during read operations. Our results indicate that an architect seeking to integrate vector data into a DBMS solution should consider the demonstrated feasibility of vector-capable single DBMS solutions in all but the most vector heavy, read intensive workloads.

Keywords: Database Management System, Vector data, Benchmark, DBMS

Sammanfattning

Bakgrund. Valet av ett databashanteringssystem (DBMS) är en komplex uppgift som ofta kräver flera års erfarenhet och en ingående förståelse av de olika tillgängliga lösningarna. I takt med den fortsatta utvecklingen inom området fördjupas problematiken ytterligare, och introduktionen av vektordata utgör inget undantag.
Syfte. För att nå en möjlig lösning på detta problem identifierades distribuerade lösningar med hög prestanda för ett antal avgränsade användningsfall. Med utgångspunkt i detta urval utvecklades ett rekommendationsramverk. Ramverket utvärderades därefter med avseende på korrekthet och användbarhet.
Metod. I denna avhandling användes Design Science Research Process (DSRP) för att vägleda arbetets övergripande struktur. En inledande litteraturstudie genomfördes för att identifiera problemet med att välja en databaslösning, varefter en andra litteraturstudie användes som stöd vid utformningen av ett ramverk för val av DBMS. Ramverket utvärderades därefter med hjälp av benchmarkdata som erhölls genom ett kvasiexperiment, och insikter om ramverkets användbarhet inhämtades genom statisk validering i form av en expertundersökning.
Resultat. En rekommendationsmetod i form av ett flödesschema togs fram. Kvasiexperimentet bekräftade delvis antagandena om påverkan av varierande kvoter mellan mängderna vektordata, samtidigt som det utmanade förväntningarna kring kapaciteten hos fristående lösningar. Enkätsvaren var generellt positiva till detta tillvägagångssätt, där en majoritet av respondenterna framhöll användarvänligheten, även om det tydliggjordes att formatet hade betydande begränsningar.
Slutsatser. Införandet av vektordata i processen för val av databas medför ytterligare överväganden, särskilt vid läsoperationer. Våra resultat visar att en arkitekt som avser att integrera vektordata i en DBMS-lösning bör beakta den påvisade genomförbarheten hos enskilda DBMS-lösningar med stöd för vektordata, utom vid de mest vektorintensiva och läsintensiva arbetsbelastningarna.

Nyckelord: Databashanteringssystem, Vektordata, Prestandamätning, DBMS

Acknowledgments

We would like to extend our gratitude to Dr. Ricardo Britto, our supervisor at BTH, for the regular feedback on thesis structure and research approach.

Further, we would like to thank our external supervisors at Ericsson: Petrit Gerdovci and Joao Felipe Ouriques, for their extensive industry insight and assistance with the creation of the framework.

Finally, we would like to thank the survey respondents for their valuable input on the recommendation framework.

Contents

Abstract i

Sammanfattning iii

Acknowledgments v

1 Introduction 1
- 1.1 Background . . . 1
- 1.2 Problem statement . . . 2
- 1.3 Aim and objectives . . . 2
- 1.4 Scope and limitations . . . 3
- 1.5 Ethical consideration . . . 4
- 1.6 Outline . . . 4

2 Related work 5

3 Method 9
- 3.1 Research method selection . . . 9
- 3.2 Research design . . . 10
- 3.3 Problem identification . . . 12
  - 3.3.1 Literature review 1 . . . 12
- 3.4 Solution design . . . 13
  - 3.4.1 Literature review 2 . . . 13
  - 3.4.2 Artifact design . . . 15
- 3.5 Evaluation . . . 17
  - 3.5.1 Quasi-experiment . . . 17
  - 3.5.2 Expert survey . . . 20
  - 3.5.3 Threats to validity . . . 22

4 Results and analysis 25
- 4.1 Problem identification . . . 25
- 4.2 RQ1 – DBMS candidates . . . 26
- 4.3 RQ2 – Selection framework . . . 28
  - 4.3.1 Artifact design . . . 28
  - 4.3.2 Expert survey . . . 28
- 4.4 RQ3 – Performance comparison . . . 30

5 Discussion **39**
5.1 Experiment results 39
5.2 Systematic selection of DBMSs 42
5.2.1 Expert survey 42
5.2.2 Experiment impact on DBMS selection 43

6 Conclusions and future work **45**
6.1 Conclusions 45
6.2 Future work 45

References **47**

A Survey responses **53**

Chapter 1

Introduction

1.1 Background

The Database Management System (DBMS) is an integral part of any non-trivial software architecture, and its ubiquity has led to the emergence of numerous categories and types. The distributed NoSQL database has become a critical component for large-scale operations, bringing substantial benefits in reliability, performance, and ease of expansion [1]. With the increasing maturity of AI technology, the need for these systems to manage both entity-related and vector data has increased [2, 3].

To reflect this, a multitude of established distributed database management systems have been equipped with native vector storage and similarity search functionality. Cassandra[1] and ScyllaDB[2] are two examples of this. In other cases, such capabilities can be found attached via extensions such as pgvector[3]. Both configurations, combined with the possibility of running multiple dedicated distributed databases simultaneously, add many permutations to an already complex database selection process [4].

To inquire into which such solution is the tool for a given workload raises a number of questions in turn. By what standard should options be selected for this use case? Which present options match this standard? Given that distributed databases have a multitude of properties with varying impacts on their capability, the selection process can be simplified by focusing purely on performance. For instance, Discord Inc. released a blog post [5] documenting the process of migrating its database from Cassandra, which the author describes as a high-maintenance system with unpredictable latency, to ScyllaDB. This migration resulted in improved uptime, increased efficiency, and reduced insert operation time – all factors of performance.

The introduction of vector data-handling functionality opens a previously unexplored domain in database selection, even when the scope is limited to performance analysis. For instance, databases are generally optimized for write-heavy workloads [6]. While the inverse also occurs with lesser frequency, there nevertheless exists a clear tradeoff between the 2 approaches [6]. When vector data is introduced, this relationship may be amplified, unchanged, or even reversed; determining *which* is one of the questions this thesis seeks to answer.

Varying workload is not the only factor in a database solution. Architectural

[1] `https://cassandra.apache.org/doc/latest/cassandra/vector-search/concepts.html`

[2] `https://cloud.docs.scylladb.com/stable/vector-search/vector-search-quick-start.html#quick-start-guide-to-vector-search`

[3] `https://github.com/pgvector/pgvector`

solutions, such as replication strategies in distributed databases, play a pivotal role in explaining performance results [7]. At a more granular level, factors such as data replication are, in turn, affected by, among other things, quorum configuration [8], update location and propagation, server determinism, and client transparency [7]. A thorough review of each option's impact on performance would allow for optimizing the most aligned database solution.

Selecting a distributed database for any given software stack is by no means a simple decision. Even after completely disregarding non-preference-based metrics, such as developer ergonomics, a plethora of choices remain [9]. It is unclear whether these principles still apply in the era of vector data. In addition, it introduces a new dimension for testing the integration of such data.

1.2 Problem statement

The problem statement discussed in this thesis has been formulated in collaboration with Ericsson[4]. The need to handle vector data alongside normal traffic at large scales has been identified, and as such, a search for capable database solutions will be conducted. Given the identified options, the problem at hand is then the discrimination between these options by systematic means.

1.3 Aim and objectives

This thesis seeks to conduct research and open the door to vector-mixed benchmarking by focusing on the narrower area of database selection within this new field. This is accomplished by creating a systematic approach to selecting a performant database management solution for a given use case. The objective is considered achieved once an unambiguous recommendation framework can be constructed. The objective is divided into sub-goals in accordance with the following list:

1. Find and compile a set of suitable DBMSs for analysis
2. Create a database recommendation framework
3. Validate the correctness of the framework in terms of DBMS performance characteristics
4. Validate the practical utility of the recommendation framework

To reach these objectives, the following research questions have been formulated:

- **RQ1** – Which distributed databases best support both read-heavy, write-heavy, and AI-vector workloads?
- **RQ2** – How can we create a useful DBMS selection framework targeting combined AI and traditional applications?

[4] `https://www.ericsson.com`

- **RQ3** – How do the selected database solutions and their configurations – dedicated, extended, or hybrid vector search – compare in terms of performance and hardware consumption?

1.4 Scope and limitations

As a complete systematic method for every DBMS solution was unrealistic, only a small subset of this area will be prototyped in the thesis. As the primary purpose of the thesis remained the creation and validation of a recommendation framework in concept, attempting to cover every use case could also result in an impractical scope.

Every DBMS solution featured in this thesis had to be either distributed by default or support such a configuration. This was due to the exponential increase in the use of NoSQL databases, driven by their horizontal scalability [10]. Due to this fact, a thesis time constraint, and the requirement of large-scale analysis prompted by the problem statement, only databases with distributed capabilities were included in the recommendation.

There are a plethora of functional and non-functional factors in any DBMS solution, most of which had to be set aside in this thesis for the sake of time and to achieve a higher accuracy of measurement. Features such as ease of deployment and query syntax may be selected in an ordinal fashion, while performance metrics such as throughput and CPU utilization are ratio data and can thus be presented, granting a greater degree of explainability to the final recommendation framework.

For ease of testing, DBMS solutions without publicly available source code were omitted from the recommendation framework. This was done to prevent potential licensing issues while also providing an additional level of transparency and, thus, reproducibility in testing.

Every tested use case would involve some degree of vector data. It was not the goal of this thesis to compare vector databases in isolation, nor to verify how traditional databases handle traditional workloads.

The verification stage of this thesis conducted tests to test a theoretical hypothesis about how these databases perform with varying amounts of vector data. This needed to be done at a reduced scale compared to what may be found in a realistic scenario. This was done primarily to sufficiently stress the database solution, even with the relatively meager specifications of the deployed benchmarking pod, which also had to run on the cluster. The reduced scale served the secondary purpose of simplifying the tuning of database resources.

Such tuning was done only to a limited extent. Many aspects of each DBMS were left to be resolved automatically by the default configuration. The performance of each database may be much greater than demonstrated in this thesis with fine-tuning, but any tweaking of values beyond external resource allocation was omitted whenever possible. This reduction in scenario realism is done in the interest of test fairness and project time constraints.

1.5 Ethical consideration

The ethical implications of this research are considered minimal and insignificant, provided that non-disclosure agreements are upheld or open-source data is available. During testing, the data structure was selected in order to mimic a realistic use case. The research has been conducted using open-source datasets, such as ArcFace, as used by Franciškovi ć et al. [11], or generated using the Faker Python package. No personal data has been collected or processed in the process of writing this thesis.

The survey segment of this thesis ran the risk of containing information that ought not to be disclosed. All participants in this survey were informed upon taking it that their answers may be featured in full in this thesis. If respondents had provided information which could be considered to be private, care would have been taken to discuss which information may or may not be published prior to doing so, and any private information would have been redacted before presenting the results.

Societal impact was considered negligible. Assuming the expected outcome will apply to a larger number of compute-heavy industries, the choices of developers, system architects, and entrepreneurs may at best marginally improve the efficiency of these industries.

There was no explicit consideration of sustainability in this study, although the nature of the expected outcome suggests that widespread adoption would improve overall processing capacity. This could marginally increase productivity or reduce power consumption in certain sectors.

1.6 Outline

The thesis follows a standard report structure. Beyond this point, the thesis consists of the parts outlined subsequently. The Related Work section is intended to provide context for where this thesis fits within the current research landscape, the general research area, and the gap(s) it addresses. The Method describes how the research artifact was produced and validated. It more specifically describes the research design methods used and how each step in this process was conducted, justifying the process from problem identification through evaluation. The Result and Analysis section describes the research outcomes and whether the artifact serves its intended purpose. The discussion chapter elaborates on the results in a broader context — what impact the findings have — and touches on how the research could have been improved further. Finally, the Conclusion and Future Work sections discuss where this research leaves the field and where future research may take place.

Chapter 2

Related work

As the need to store vector data increases, the mixed-workload scenario will become more common. To properly assess the intersection of these fields, a brief retrospective of both vector database research and performance-based DBMS selection methodology is required.

Wiesmann et al. [7] present a number of parameters for performance study, specifically for distributed systems such as synchronicity and quorum configurations. The report identifies key replication techniques and their trade-offs. They conclude that further research into the performance of the different approaches is needed, while providing important groundwork for performance testing of distributed systems.

Wiesmann and Schiper [12] provide a theoretical comparison using pseudo-code in a follow-up study. They conclude that total order broadcast-based replication techniques significantly outperform methods such as primary copy, distributed locking, and lazy update-everywhere replication. The methodological approach is far more theoretical than the practical focus attempted in this thesis.

As an introduction to vector database concepts, Pan, Wang, and Li [3] provide an overview of many such systems and lists several databases that support some of these functionalities. Additionally, the authors discuss query optimization and several key benchmarks for experimental comparisons.

In subsequent research, Wang et al. [13] developed the Milvus database, and the authors discuss several vector-specific optimization techniques the database employs. Their research helps form a complete picture of how a vector-exclusive database may differ from a hybrid database. While approaches such as expanding the AVX512 instruction set or adding multi-GPU support may not be novel, they are a boon to the vector-exclusive database approach. Such mechanisms, however, cannot always be tested, which highlights potential blind spots in testing.

Nicola and Jarke [14] provide a more general, principled analysis in which the fundamentals of data replication are discussed. The authors discuss data access pattern assumptions, inter-database communication, and transaction processing models, among other topics. The authors stress the importance of studying systems in their entirety, noting that many models either isolate the database or the network, neglecting the system as a whole.

On the specialization of databases, Stonebraker and Cetintemel [4] note the lack of any "one size fits all" solution to the DBMS question. Instead, as data diversity increases, the authors state, it will become necessary to adapt such choices on a per-system basis. This is congruent with the problem this thesis seeks to solve.

The prior study has a follow-up by Stonebraker et al. [15], who benchmark DBMS

solutions, further proving the previously demonstrated design pattern. The article lists several performance factors that are significant in traditional databases – factors that can be extrapolated to, at the very least, distributed systems, and perhaps partially to vector databases.

In a similar vein, Dewitt et al. [16] take an early look at methods for database benchmarking, especially in a multi-user environment. Although dated, the usage of throughput as an important metric remains relevant, and a similar methodology is applicable to this thesis.

The highly legible and simple benchmark of 3 database systems conducted by Ohlsson et al. [17] provides a template similar in scale, methodology, and motivation to this thesis. The authors test each database across 6 different workloads, each with varying read, write, and scan rates, using YCSB. Critically, the reads and writes are part of the same test, distributed by percentage. It differs in testing in the lack of sustained load; something this thesis attempts to remedy.

Taipalus [18] discusses a number of growing pains found in vector databases. Among these is the balancing between speed and accuracy. Since vector databases handle high-dimensional data, they are often abstracted to improve performance. As abstractions go, an unavoidable trade-off is the query's accuracy. Furthermore, as these vectors grow in size, as they need to contain more and increasingly complex data, the above pain-point will grow greater, not lesser, with the passage of time. Finally, Taipalus brings up the issue of the field's overall maturity. As the field is novel, it may change entirely at any time. This can potentially uproot any selection process or existing software stack.

In a more recent literature review, Taipalus [19] outlines a collection of benchmarks and a general comparison of SQL, NoSQL, and NewSQL systems. The paper concludes that although there are plenty of DBMS performance comparison studies, they note insufficient detail for reproducibility. Furthermore, the article presents a collection of considerations for the industry given these traditional non-vector workloads.

There are numerous potential pitfalls in benchmarking, which are comprehensively covered by Raasveldt et al. [20]. Although inheriting core concepts such as "apples to oranges" comparison from the basics of experiment design, the author delves into greater detail. Overly specific tuning, optimization neglect, and incorrect database code are explicitly accounted for when designing the benchmark and analyzing the results of this thesis.

On the subject of general database selection methodology, Zahedi [21] presents a method that uses a matrix populated with values representing the number of supported primitives. Primitives, which are the basic functions a database should perform, e.g., inserts, updates, or deletes. While this example method is generally not needed, since most basic primitives are supported in modern databases, it is nonetheless a useful approach. In comparison, the framework employed in this thesis aims to be simpler in nature and easier to follow as a quick reference, rather than as a calculation exercise.

Brahimi et al. [22] presents a similar, but modernized approach. The author proposes a method in which deployment information is specified, and a large benchmark dataset, combined with machine learning, answers potential database users' inquiries. The method is highly quantitative and requires a large dataset to function

properly. In contrast, the system proposed in this thesis aims to start small in scale, with potential for additive growth.

Yet another, entirely novel parameter-based approach is presented by Mateen et al. [23]. The authors analyze 3 databases in terms of their autonomic computation characteristics, specifically their self-configuration, that is, how well the autonomic systems analyze and adapt to the technical specifications of their hosts. This procedure culminates in a table of floating-point values, in which the scores for each autonomic system are presented, from which values are taken for 3 different use cases. Although this research will not be used directly in this thesis, as autonomic systems are far beyond its scope, the floating-point scoring system, as applied to any set of use cases, is an even simpler selection method when dealing with a large number of metrics.

In summary, we identify a set of parameters for database comparison in both distributed and non-distributed systems [7, 12, 16, 18]. The same is found for vector databases [3]. The performance metrics discussed are indeed the same as those relevant to our research, although no comprehensive recommendation framework has been developed. Similarly, the literature covering the implementation aspects of vector databases and the principles of distributed databases in greater depth points to several additional optimization metrics worth considering, even though time constraints will certainly prevent comprehensive coverage [13, 14]. Regardless, such metrics are of use for making informed decisions during the experiment.

Separately but related, we find a set of standards which ought to be followed during the experiment, lest the result be invalid [20]. The studied works also provide guidance on how databases compare. The similarity to our research lies in the use of direct, research-based benchmarks, although neither article discusses vector databases [15, 19]. We find 2 distinct potential solution approaches for the artifact design are presented [21, 22], and a third, simplified approach [23]. Finally, the literature on database specialization illustrates that database selection is by no means unimportant [4]. Despite the best attempts to find a universal solution, database specialization remains of great importance. This relates to our research by directly pointing to the gap. As the literature concludes, the mention of emerging applications and new DBMS solutions links directly to the emergence of vector databases.

Chapter 3

Method

3.1 Research method selection

The three research methodologies described by Wohlin and Runeson [24] are Design Science Methodology, Action Research, and Technology Transfer Research Methodology. According to the authors, the primary focus of Design Science is to design and implement a solution or artifact that solves a problem in a particular context. The solution or artifact is then evaluated on how well it addresses the problem. Design Science requires a well-known problem and access to enough theory to inform the design.

Meanwhile, the authors describe Action Research as investigating real-world problems concerning processes and ways of working, and applying an intervention to improve the situation. The results following the intervention are then evaluated to determine if the intervention had the intended effect. Action Research requires a real-world organizational problem as well as a real-world scenario or organization to enact the change upon, and thus requires very close collaboration with industry practitioners.

Lastly, the authors outline the Technology Transfer Research Methodology, which is focused on transferring and implementing theoretical solutions from the academic world in real-world industrial contexts. It requires close academia-industry collaboration, but slightly less so than Action Research.

Design Science was ultimately chosen as the research methodology to be used in the thesis, because the main contribution of this research was an artifact, and not an organizational change or the application of already existing academic theory in a real-world scenario.

The objectives were reached by following the Design Science Research Process (DSRP) [25], adapted to our needs. It was deemed that this process would allow for a systematic methodology, which befits a larger framework that can contain both the creation and evaluation of an artifact. The modifications made were a mere streamlining, as well as a substitution of the laboratory experiment for a quasi-experiment. This streamlining was done as the nature of this project does not lend itself to interviews, or case studies. These aspects of the methodology were therefore omitted. The authors explain that the DSRP can optionally be performed iteratively to improve the artifact over time. This thesis iterated on some of the phases before moving on to the next, but did not return to earlier phases because of the time constraint.

As the DSRP includes two distinct literature reviews, employing the Rapid Re-

view method [26] helped keep the reviews expeditious. This allowed more time to be allocated to the experiment and provided greater accuracy where it mattered most. A more thorough, systematic literature review process might have provided greater confidence and rigor; however, it would also have taken more time, which was a limited resource. Additionally, the more limited literature review process was considered adequate for this project.

A quasi-experiment was chosen as a method to support the evaluation process as the conducted experiment would not benefit from randomization [27]. Unlike a standard variant of such an experiment, randomizing subjects (i.e., databases) or treatments (i.e., performance tests) would do little to solve any problems. Quasi-experiments, in principle, require no randomization [25], which suited the more deterministic nature of dependent variables [27].

An expert survey was used as input when evaluating the created artifact. The survey provided new insights into the artifact's practicality and utility, while the quasi-experiment provided information only on the correctness of the artifact's recommendations. Additionally, the DSRP recommends conducting an expert survey as part of the evaluation phase.

3.2 Research design

The Design Research Process (DSRP) [25] was chosen as the overarching research framework for the thesis. The different phases and activities of the DSRP utilized in this report are visualized in Figure 3.1 and are briefly described below:

- **Problem identification**
  - Literature review 1: To support the problem identification phase, the first literature review was conducted. The objective was to find sources that highlighted the importance of selecting an optimal DBMS for a given system, the difficulty of selecting a DBMS, and whether any research had been conducted on the systematic selection of a DBMS.
  - Identify problem: The problem was identified with the results from the first literature review and refined during the whole problem identification phase.
  - Pre-evaluate relevance: The relevance of the problem was evaluated and mapped to the solution space.
- **Solution design**
  - Literature review 2: This literature review was conducted during the solution design phase. The focus of this literature review was to find DBMS benchmark comparisons to guide the design of the recommendation framework.
  - Design artifact: During this phase, the recommendation framework itself was developed.
- **Evaluation**

– Refine hypothesis: The main hypothesis was split into smaller, more concise hypotheses to facilitate evaluation through the quasi-experiment and expert survey.

– Quasi-experiment: The objective of the quasi-experiment was to evaluate whether the results of literature review 2 are still true in this scenario, and by extension, if the artifact that was created needs to be revised.

– Expert survey: A limited expert survey was conducted among Ericsson software architects as a way to qualitatively gauge experts' opinions about our solution to the problem.

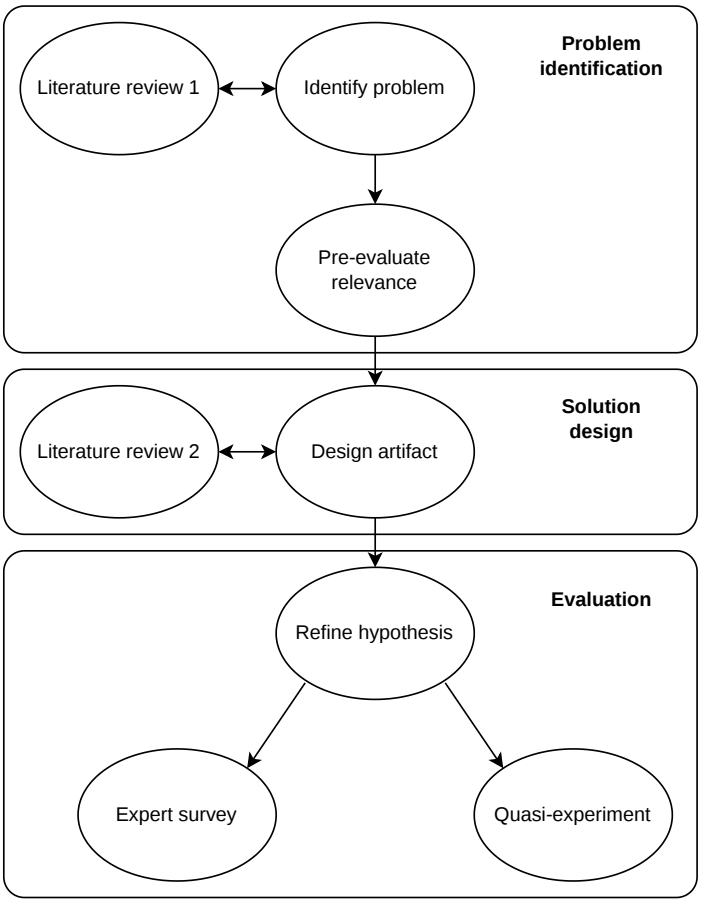


Figure 3.1: The different phases and activities of the Design Science Research Process (DSRP).

3.3 Problem identification

3.3.1 Literature review 1

A streamlined version of the Cochrane rapid review technique described by Devane et al. [26] was used in both literature reviews to limit time expenditure while retaining as much scientific rigor as possible. The first literature review was conducted to explore the problem of selecting a database management system for a given software stack, with a focus on optimizing non-functional requirements such as performance and resource utilization. The databases and search terms utilized in the first literature review are presented in Table 3.1.

Table 3.1: Search terms and databases used in literature review 1.

Database	Search term
ACM Digital Library	[[Abstract: "performance"] OR [Abstract: "efficiency"] OR [Abstract: "benchmark*"] OR [Abstract: "hardware utilization"] OR [Abstract: "resource utilization"]] AND [[Abstract: "database"] OR [Abstract: "dbms"] OR [Abstract: "database management system"]] AND [[Abstract: "software stack"] OR [Abstract: "software architecture"] OR [Abstract: "application stack"] OR [Abstract: "technology stack"]]
Scopus	TITLE-ABS-KEY ("performance" OR "efficiency" OR "benchmark*" OR "hardware utilization" OR "resource utilization") AND ("database" OR "DBMS" OR "database management system") AND ("software stack" OR "software architecture" OR "application stack" OR "technology stack")
IEEE Xplore	("All Metadata":"performance" OR "All Metadata":"efficiency" OR "All Metadata":"benchmark*" OR "All Metadata":"hardware utilization" OR "All Metadata":"resource utilization") AND ("All Metadata":"database" OR "All Metadata":"DBMS" OR "All Metadata":"database management system") AND ("All Metadata":"software stack" OR "All Metadata":"software architecture" OR "All Metadata":"application stack" OR "All Metadata":"technology stack")
Google Scholar	systematic dbms selection

After querying the databases, the selection process began. First, the sources were deduplicated on a title basis using a Python script, then excluded via AI analysis of the titles. Grok[1] 4.2 was supplied a text file with the de-duplicated titles and prompted to *"Remove titles that are irrelevant to the subject of selecting a database for a software stack."*. The AI's work was briefly verified by checking some of the removed titles to ensure they did not contain titles relevant to the research. This was

[1] `https://grok.com`

done until a point which was satisfactory to the authors, with no well defined lower or upper bound. The rest of the selection process was done manually as the number of sources had become more manageable. One author then reviewed all remaining titles, selecting those relevant to the study. The second author then reviewed a random 25% sample of the titles that the first author reviewed. The decisions were compared and discussed. The authors agreed on approximately 95% of decisions.

Given the limited number of sources remaining after the title-based exclusion, the authors collaborated to discuss and exclude sources based on abstracts and full-text reads. Following the manual full-text reads, the decision was made to conduct a quick complementary Google Scholar search to identify sources related to selecting a DBMS for a given set of requirements. The authors reviewed a limited number of results and excluded the irrelevant ones on the spot. The basic criteria for inclusion in literature review 1 were the following:

- The source must be written in English.
- The source must be available to read online with a BTH student account.
- The source must be relevant to selecting a DBMS and its impact on the performance of the software stack.
- The source must follow a correct, largely reproducible methodology with relevant sources.

The selection process, along with the number of remaining sources after each step, is shown in Figure 3.2.

3.4 Solution design

3.4.1 Literature review 2

As part of the solution design phase, a second literature review was conducted. The objective of this literature review was to discover DBMS performance data to use as a foundation for the initial database recommendation artifact, which was reflected in the chosen search terms presented in Table 3.2. The decision to add the Google Scholar search comparing Cassandra and Scylla was made for its inclusion in the Discord blog [5], where it is cited as a viable replacement for Cassandra. It was also deemed that the addition of another database with a hybrid vector approach would be suitable for the experiment.

The selection process depicted in Figure 3.3 was very similar to that of the first literature review, detailed in subsection 3.3.1. The most significant difference lies in how the AI-powered title filtering was performed. The number of sources was more than three times as many as in the first literature review, and the AI tool could not handle the large list in one pass, so a Python program was created, which iterated over the list of titles and prompted a local AI model with the following: *"Answer only 'Yes' or 'No'. Does the following title describe a report about benchmarking a Database Management System? <title>"*, where <title> was replaced by the actual title of the source. Titles that received a 'Yes' answer were kept, and those that

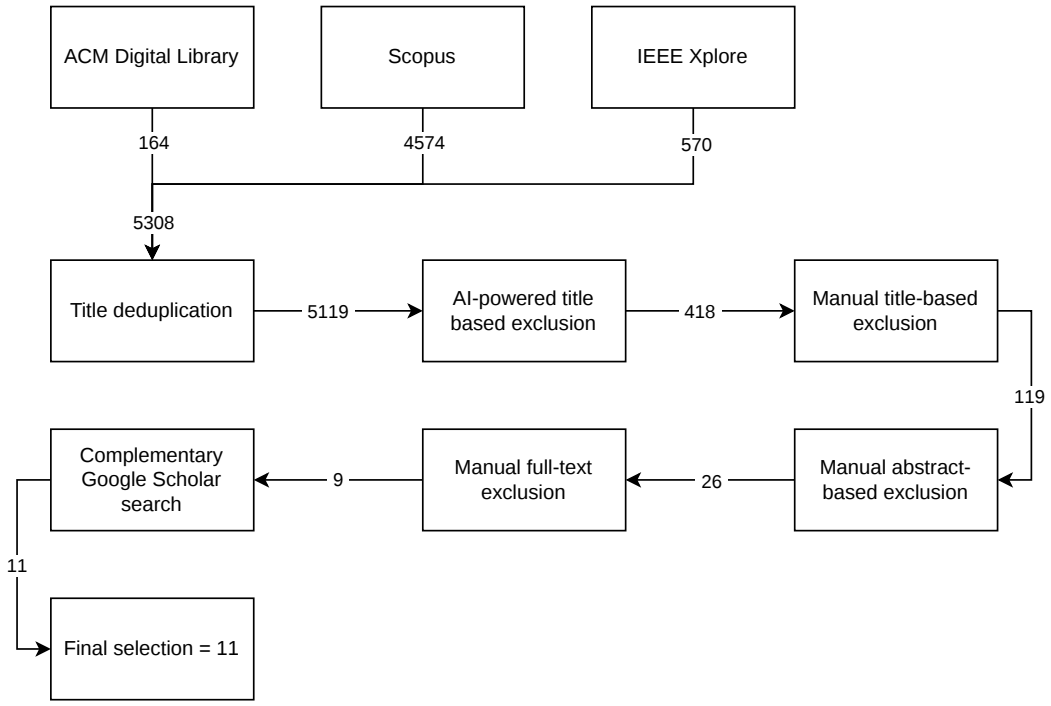


Figure 3.2: Literature review 1 process with the number of remaining sources after each step.

received a 'No' were discarded. The rejected titles were reviewed briefly to verify that no obvious mistakes had been made. Much like the process in the first literature review, no concrete number of reviewed titles were recorded, but enough titles were reviewed to satisfy the authors. Ollama[2] version 0.18.0 was used to run AI models locally, and the model used to filter the sources was Qwen3.5:9b[3]. The rest of the selection process continued as in the first literature review.

After the selection process was completed, the contents of the remaining sources were extracted into a spreadsheet with the following fields:

- Source ID and link to the PDF
- Theme of the source
- List of which database management systems were tested
- Key findings
- Methodology used
- Miscellaneous notes about the source or useful quotes

After the sources' contents were extracted in a standardized manner, a brief quantitative analysis was conducted. Since the objective of the literature review was

[2] `https://ollama.com`
[3] `https://ollama.com/library/qwen3.5:9b`

Table 3.2: Search terms used in literature review 2.

Database	Search term
ACM Digital Library	[Abstract: performance] AND [[Abstract: database] OR [Abstract: dbms] OR [Abstract: database management system] OR [Abstract: vdbms] OR [Abstract: vector database]] AND [Abstract: comparison]
Scopus	ABS (("database" OR "dbms" OR "vdbms" OR "vector database") AND "comparison" AND "performance") AND (LIMIT-TO (SUBJAREA , "COMP"))
IEEE Xplore	("Abstract":performance) AND ("Abstract":database OR "Abstract":dbms OR "Abstract":database management system OR "Abstract":vdbms OR "Abstract":vector database) AND ("Abstract":comparison)
Google Scholar	cassandra scylla dbms

to guide the creation of the artifact, the most analytically relevant data were the relative performance of the different database management systems. The databases were awarded points for every other database they outperformed in the sources. In a benchmark comparing three databases, the most performant database received 2 points because it outperformed the other two; the second received 1 point; and the third received none. Points were also awarded in the different categories *read*, *write*, *update*, and *delete*. This is not a perfect metric by any means, as it allows a single source to award excessive points to a single database, and more commonly featured databases would likely receive higher scores than more performant but lesser-known ones. However, it was deemed a practical way to present the results of the literature review. Further, to mitigate the latter potential problem, the points were normalized by dividing by the number of reports in which the databases were featured.

3.4.2 Artifact design

The options available in the initial artifact were selected by studying the data visualized in Figure 4.2 and Figure 4.1. By researching the features and characteristics of each remaining DBMS, these could then be sorted into categories. Through reexamination of the sources in the literature review, performance indicators, such as ACID compliance, were formulated into questions. General DBMS tradeoffs were also noted, such as strong read performance at the cost of write capacity. The resulting draft was then iteratively refined based on feedback from the external supervisors at Ericsson.

Specifically, this feedback consisted of alterations to the posed questions, as these, though derived from the literature review, could be phrased in a multitude of ways and lacked systematic justification. Inclusions from this feedback did not constitute the addition or removal of databases unless they could also be clearly motivated by

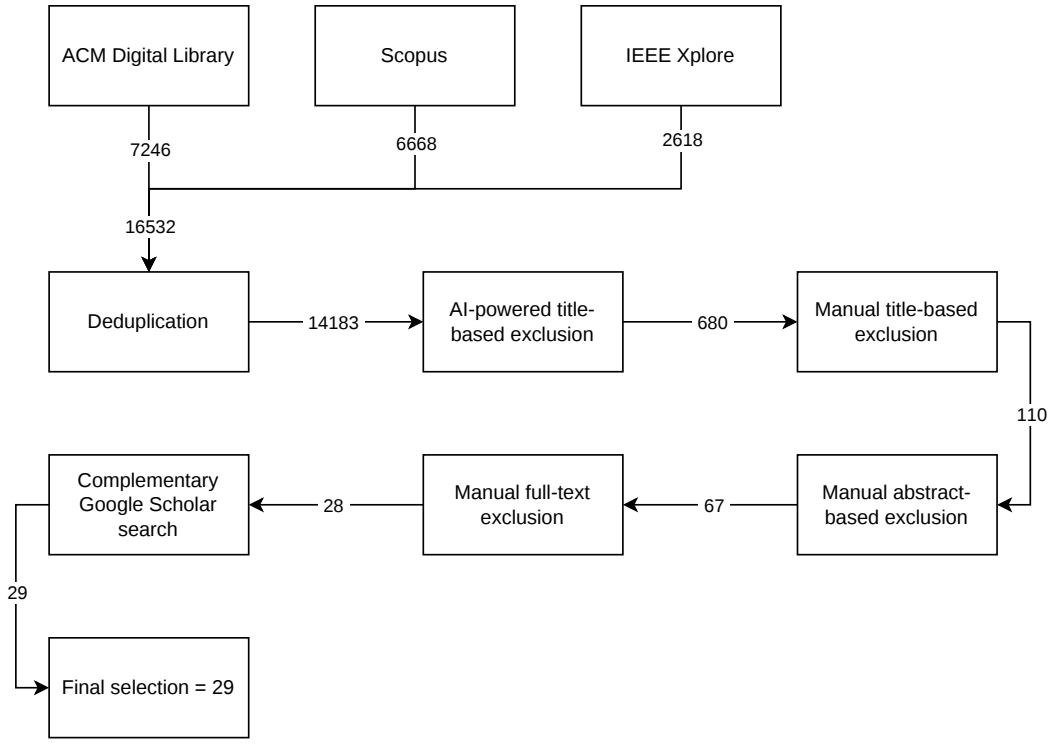


Figure 3.3: Literature review 2 process with the number of remaining sources after each step.

the literature review. For example, the initial selection of databases was broader. NuoDB was removed in later iterations because its code availability was erroneously deduced from the available literature to be free. The state of the code would have been the case in the literature also, thus justifying its removal. In contrast, VoltDB discontinued the community edition of its DBMS in January of 2025[4]. This event took place after the relevant literature was published, and thus VoltDB could not be removed from the artifact.

The impact of vector data in a mixed environment was both unknown and untested at the time of the literature review. Thus, the recommendations featured in the artifact are a pragmatic estimate of reality, based exclusively on the isolated performance of pgvector [11].

In events where insufficient information was available to meaningfully discriminate between 2 DBMS solutions, no such information was separately sought out. Rather, the results of the literature review were considered frozen upon acquisition and "statically" analyzed. This methodology, though the cause of erroneous inclusions, was pursued regardless to increase reproducibility.

[4]https://www.voltactivedata.com/blog/2025/01/introducing-the-volt-active-data-developer-edition/

3.5 Evaluation

3.5.1 Quasi-experiment

To evaluate the correctness of the artifact, a quasi-experiment was conducted. In practice, this experiment consisted of a series of DBMS benchmarks with different parameters. These parameters were read/write balance, vector/non-vector balance, and the usage of a dedicated vector database or built-in functionality in the databases that supported it. The database management systems under testing were MySQL NDB Cluster, MongoDB, Cassandra, Scylla, and Milvus.

3.5.1.1 Null-hypothesis

The experimental design was structured around three null-hypotheses, namely:

- The choice of database has no statistically significant impact on hardware utilization.
- The choice of database has no statistically significant impact on traffic throughput.
- The choice of database has no statistically significant impact on latency.

3.5.1.2 Variables

The subjects of the experiment were the databases under test and the objects were the different traffic types. The traffic types were defined as read-heavy or write-heavy as well as vector-heavy or vector-light. The read-heavy traffic type was defined as being 75% read and 25% write traffic. The opposite is true for the write-heavy traffic type. Similarly, vector-heavy traffic consisted of 75% vector traffic and 25% regular traffic, and the opposite was true for vector-light. The independent variables comprised databases, testbed setup, database configuration, traffic type, and data structure. The testbed setup was kept constant between benchmarks and the database configurations in terms of resources were kept fair to not give any database an unfair advantage. The dependent variables, which were monitored to assess system behavior, comprised traffic throughput, latency, as well as CPU and memory consumption.

3.5.1.3 Sampling strategy and treatment assignment

Following the second literature review detailed in subsection 3.4.1, suitable DBMS alternatives were selected from the ones presented in the analyzed works. Adhering to the theme of the thesis, only databases that were horizontally scalable, open-source, and locally deployable in a Minikube[5] cluster were selected. This, being a quasi-experiment rather than a traditional experiment, relied on purposive sampling rather than randomized sampling. Every database management system that appeared in the second literature review and fulfilled the criteria was tested as part of the benchmark.

[5] `https://minikube.sigs.k8s.io`

The treatments were assigned according to a fully crossed within-subjects design, meaning every DBMS solution was tested on every traffic type. The 6 database combinations tested were Cassandra-Milvus, Cassandra-Standalone, MySQL-Milvus, MongoDB-Milvus, Scylla-Milvus, and Scylla-Standalone. Given that Cassandra and Scylla had built-in vector support, they were both tested standalone and in combination with the Milvus vector database to allow for comparisons between hybrid and dedicated vector databases. MySQL and MongoDB did not have built-in support for vector data at the time of writing, so they could only be tested alongside Milvus to support the traffic patterns. The 4 traffic patterns were write-heavy-vector-heavy, write-heavy-vector-light, read-heavy-vector-heavy, and read-heavy-vector-light. This combination of 6 DBMS solutions and 6 traffic types yielded a total of 24 test cases. The databases were cleared between executions to reduce the problem of carry-over effects.

3.5.1.4 Data analysis

Utilizing various python scripts the output data was codified as CSV files. As the generated data was multi-threaded, the number of entries varied per thread. In these cases the output was trimmed to the fewest entries. The processed files were then imported into a spreadsheet, from which the data was finally exported into graphs. In order to make data more legible, presentation was made logarithmic when appropriate.

3.5.1.5 Benchmarking design

The reason only a single vector database is tested is a matter of permutation. To maintain thesis legibility, the number of databases was reduced, and Milvus was chosen as the sole "champion" due to its performance results in [11]. It is for this reason vector search accuracy will not be evaluated in this thesis.

The purpose of the benchmark testing was to validate the designed artifact using our own performance measurements. This helped verify that the relative performance of the different databases was the same as reported in the reviewed sources, as well as to test databases that had not been compared in the sources. Additionally, the benchmarks sought to provide new insights into DBMS performance given a mix of vector and non-vector data. The DBMS recommendation framework was based on the assumption that a database that supports both vector and non-vector data is able to handle small amounts of vector data without a performance degradation, while a system that is expected to handle a larger amount of vector data would benefit from utilizing a dedicated vector database.

After exploring different options of DBMS benchmarking tools, such as HammerDB[6], JMeter[7], Yahoo! Cloud Serving Benchmark (YCSB)[8], BenchBase[9], etc., the decision was made to implement a custom benchmarking program. This was due to a lack of functionality required for this experiment, such as support for all the

[6] `https://www.hammerdb.com`
[7] `https://jmeter.apache.org`
[8] `https://github.com/brianfrankcooper/YCSB`
[9] `https://github.com/cmu-db/benchbase`

databases being tested and ramping intensity across different distributions of read and write traffic.

This custom benchmark was implemented to be minimally viable for completion of the experiment. It consisted of 3 components: A data generator, a collection of database abstractions, and finally the ramping curve loop.

The data generator used Faker[10] and the large-arcface-512[11] dataset to prepare data for insertion. It did this using Faker and Apache Pyarrow[12]. The program is multi-threaded in a relatively hard-coded fashion, expecting the total number of threads to be divisible by 10. Each of these threads generated its own data using this component, then waited for the others to do the same via a barrier, after which the benchmark proceeded to each database's abstraction.

Based on the databases selected via runtime flags, a custom class that matched them was selected. This class, regardless of the database, had a specific set of functions, creating a layer of abstraction between the benchmarks' ramping curves and the Python driver for each database. Some notable functions found within each database are as follows:

- init () - Ran several first-time setup steps, such as creating a database, table, or adding an index if necessary.

- generate_queries() - Acted as a continuation of the data-generation component. The generated data was prepared for insertion by creating read, write, vector-read, and vector-write queries. The proportion of these was selected by the flags set at runtime. It should be noted that a given read/write distribution applied to both the standard data and the vector data. As such, a read-heavy test, for instance, would also include a proportionally large amount of vector searches.

- execute() - A simple function which called each database's equivalent of executing a query. In certain cases, this functionality was split into 2 or more functions for the database, i.e, an explicit insert () or query() function. In these cases, each query from the prior function returned a tuple in which the first element was a short identification string. This string identified the operation as read, write, vector-read, or vector-write, and then called the appropriate function.

  It could be argued that this slightly biased the benchmark towards databases which do not require such a check, due to the additional branching introduced by the string comparison. Although this was partly true, any realistic application of such databases in which the incoming request types are unknown in advance would still require such a check. Alternatively, the additional overhead of creating multiple read-only or write-only lists of queries, which must be evenly distributed across threads, could compromise the benchmark's throughput.

  The execution implementation was further complicated by the varying conditions across databases. For example, it would be entirely possible to store

[10] `https://github.com/joke2k/faker`
[11] `https://github.com/chenggongliang/arcface`
[12] `https://arrow.apache.org/docs/index.html`

and search vector data in databases that did not support this feature, namely MySQL and MongoDB (local) in this benchmark. These fields were, in principle, no different from any long column data. Where databases like Milvus differed was in their similarity search; a much more dynamic method. It was determined that merely storing vector-like data in these databases would not adequately support the workloads typically associated with a vector database; MySQL and MongoDB were tested only in conjunction with a dedicated VDBMS.

The final component of the benchmark was the actual ramping curve. This was a simple loop that ran on each thread. Each thread attempted to push a number of queries, such that their sum reached a steadily increasing ceiling defined by constants. Operations were spaced out over a full second, attempting to complete all writes within that time. The next second, the ceiling was increased by a count specified by constants. A list of values, each representing seconds, could also be specified. These values represented thresholds at which the benchmark would plateau for the specified time. The ramping curve would continue until a specified maximum value is reached, concluding the benchmark. All source code used throughout this thesis, including this benchmark and utility scripts for literature reviews, is available on Gitlab[13]

The benchmark program and the databases under test were executed on Minikube on a machine equipped with an AMD Ryzen 5900X 12-core CPU and 64 GB of memory. The benchmark container was allocated 10 CPU threads and 20 GB of memory. Since the goal was to benchmark distributed databases, they were all deployed with 3 replicas. The database deployments were allocated a total of 6 CPU threads and 15 GB of memory. When Milvus was deployed alongside another database, they were each allocated half of those resources. Some databases had components that did not directly impact performance, such as configuration services. Those nodes were deployed without the resource limitations.

Each database solution was tested for 4 different workloads, namely combinations of read- and write-heavy and vector-light/heavy. During this runtime, the execute() function mentioned earlier recorded each operation to a CSV output file, including a timestamp with 6 decimal places and a latency measurement with 18-20 decimal places. Hardware utilization was measured using kubectl top. For each solution, the utilization of all nodes, i.e, both Milvus and its partner database, would be added together, and their sum would be regarded as full resource utilization. For all measurements in this experiment, the ramping curve was incremented by 100 ops/s every second. Plateaus would last 60 seconds, and occur every 10, i.e, at thresholds of 1000, 2000, 3000, etc. The highest number of operations per second tested was 8000, during which a final 60-second plateau occurred before the benchmark concluded. Including data generation and vector codification, each run took roughly 12 minutes, with 1 run being conducted per test.

3.5.2 Expert survey

To further evaluate the created artifact, static validation [28] was conducted through an expert survey. The survey consisted of 5 questions with free-text responses and

[13] `https://gitlab.com/brolas/master-thesis`

was distributed to 4 software architects at Ericsson with the help of our external supervisors. Respondents were not picked at random, rather, qualified respondents were hand-picked.

Given that the expected amount of responses was low, the aim was to generate qualitative responses rather than quantitative. This was achieved by using free-text responses rather than yes/no or Likert-scale questions. Such questions would require a larger sample size to yield reliable results and would also limit nuanced expert responses.

The five questions were ordered from least specific to most specific, and included a catch-all question at the end. The questions were worded to elicit verbose answers. The survey was prefaced with an introduction stating the purpose of the survey, how the answers would be used, how we would like respondents to answer the questions, and, finally, a glossary explaining the abbreviations used. The respondents were then presented with the artifact and the questions. No questions were marked as required, as that might force a respondent to answer a question they didn't understand or that they had already answered elsewhere. The questions used in the expert survey are listed in Table 3.3. The survey was open between April 25th and May 10th.

Table 3.3: Open-ended questions used in the expert survey.

No.	Question
Q1	Is this type of tool a sufficient solution to the problem of selecting a performant DBMS configuration for mixed vector and non-vector workloads? What are the strengths and weaknesses to this flowchart approach?
Q2	Is the framework in its current iteration useful, i.e, does it assist in narrowing selection or exploring options?
Q3	Is the process of following the chart intuitive? For example, is the design language familiar, and is it clear that a vector/non-vector combination of databases is possible?
Q4	Are there any DBMS options or selection branches you consider to be missing? If so, which ones do you feel should be added and why?
Q5	Is there anything else about the framework that is either well executed or in need of revision?

The survey instrument was deliberately designed to elicit rich qualitative responses from participants. A high degree of formalism in analyzing these free-text answers would have risked treating hypotheticals as statements of fact, reading sarcastic remarks as genuine, or otherwise dropping essential context [27]. Prior to analysis, no formal hypothesis was formed regarding survey outcomes. Instead, the responses were examined with the goal of such a hypothesis being generated. This is generally done through a constant comparison or other such technique [29]. Since the scope of the survey was considered very small, extraction of codes from such an already small dataset was deemed to reduce it with little benefit. The responses would as such instead be summarized narratively, with the goal of preserving their

essence.

3.5.3 Threats to validity

3.5.3.1 Reliability

The literature reviews have had numerous steps taken to increase reproducibility, such as providing the scientific databases as well as search queries used. Additionally, the process used to eliminate sources was also thoroughly documented, though the use of large language models in the filtering process introduced a degree randomness.

The quasi-experiment, and by proxy the benchmarking software remains reproducible insofar as the project is available on the associated GitLab and remains functional. The repository contains the benchmark code as well as required dependencies and their versions. The procedure itself is also described in detail in the report. Finally, the acquisition of hardware, or a similar software environment to the test bench is acknowledged as being a potential hindrance, however the relevant parts of this system are well documented, and such difficulties are the case for any benchmarking endeavor.

Regarding the survey, the nature of the medium causes an inherently lower degree of reproducibility with this few participants. The results are however entirely transparent, as the questions and answers are both available in full in Table 3.3 and Table A.1 respectively.

3.5.3.2 Construct validity

The experiment conducted during this thesis relied on a custom implementation of a benchmark rather than a standardized option. While none of the examined alternatives would have been appropriate for this benchmark, most of them are certified for conducting scientific experiments with, and are undoubtedly more well-tested and proven than our custom implementation. Throughput is in this thesis defined as the number of operations performed within a given time frame. Latency is defined as the time taken from just before a query is executed to once a result is returned by the database driver. It is plausible that that these definitions stray from the standards applied elsewhere in the industry.

It is additionally possible, and in fact likely, that the choice of Milvus as the single representative to all vector databases generalizes poorly into other VDBMS implementations. Such cases ought to be tested separately.

3.5.3.3 Internal validity

The authors deem the internal validity of the experiment was generally high. The testbed was held static over the 2 days of running tests. The caveat to this is driver implementation varying between database solutions. Depending on whether multi-threading is expected to be explicitly implemented by the benchmarker or present implicitly in the database has not been taken into account. All databases are treated "equally" by the multi-threaded benchmarking software. All drivers which had asynchronous execution had this feature utilized, whereas those without relied on the multi-threaded request-stream.

Though rebooted, the Gentoo Linux system was frozen in terms of binary version, controlled in terms of running services, and had no software capable of substantially impacting performance running during any of the tests. To assert this, real-time system monitoring programs were used during runtime. Database configuration was kept constant with the exception of allocated resources. The dataset was identical between each run, and deterministically determined by a set seed.

A further concern is that the deployment configuration of one or more database may have been inadequate, and thus unfairly configured in one database over another, misrepresenting their relative ideal performances. Since the database solutions were purposely not fine-tuned beyond basic configuration, this might also have misrepresented reality somewhat, since production databases are commonly tuned for their intended workloads.

3.5.3.4 External validity

The final selection of tested systems was a narrow set of databases, which runs the risk of reducing the generalization of this thesis result. It is plausible that all of the selected options share characteristics which would not apply universally under more inclusive criteria.

The generated synthetic data was limited in size, both from the Faker library and the Arcface dataset. Although the datasets used in the experiment was vaguely inspired by a telecom workload, produced with help from our industry supervisors, it is likely that real-world datasets would look different from what was tested in the experiment.

The results are further tied to the specific hardware and software configuration used. It is not necessarily generalizable to other CPU architectures, Kubernetes distributions or operating systems.

The observed survey results may also not be generalizable to the industry as a whole. Due to the narrow selection of participants, and their relative industrial proximity, their conclusions may be biased toward a specific solution space, which either does not exist, or in another way lacks relevance in more general cases.

Finally, as the field of vector databases evolves, the results observed during this thesis may become irrelevant, antiquated or misleading. This is the nature of a rapidly evolving field.

Chapter 4

Results and analysis

4.1 Problem identification

Two different systems for recommending or selecting database systems are presented in [21, 22]. Although [21] was published in 1985, the author highlights many contemporary issues one faces when selecting a database today, namely the sheer number of databases to choose from, the different types of databases suited to different purposes, and the lack of systematic methods for comparison. There was also an emphasis on the high cost of acquiring a DBMS at the time of writing. Decision analysis is used in the report to compare different database systems based on their core functions and the extent to which they support the decision-maker's objectives.

In a more recent paper from 2016 [22], the author notes that there are hundreds of different DBMSs to choose from and the choice will have an impact on the satisfaction of non-functional requirements. To facilitate database selection, the author proposes a system that stores benchmark data and allows the decision-maker to input information about their deployment, including expected data types and available system resources. The system will then fill in missing parameters, such as DBMS, using machine learning methods, such as similarity search or regression models.

According to [10, 30, 31], the choice of which DBMS to use in a system is far from trivial and depends on many factors. The sources compare the advantages and disadvantages of SQL and NoSQL databases. All three sources agree that there is no single solution that performs best across all applications. Samarta et al. [30] highlight the strengths and weaknesses of SQL and NoSQL databases. For example, SQL databases use standardized SQL queries, have rigid predefined schemas, support ACID transactions, and can perform operations such as joins on the data. Meanwhile, NoSQL databases are more flexible and able to handle unstructured or semi-structured data, horizontal scalability, and high availability. Many of the same advantages and disadvantages of NoSQL are discussed in [32]. As per [33], the choice of database is not just between SQL and NoSQL either. The authors of that article found that the most suitable database system for high-velocity, real-time analysis of time-based data was a time-series database (TSDB).

In real-world applications, the choice of DBMS has tangible consequences for non-functional metrics such as performance. Béjar-Martos et al. [34] compare the performance of Cassandra, MongoDB, MySQL, and PostgreSQL for storing and processing LiDAR datasets. The authors found that the NoSQL databases Cassandra and MongoDB performed better than SQL alternatives for this type of work. In an Industrial IoT context, [35] also found that the NoSQL database AWS DynamoDB

had lower latency and power consumption than the SQL alternative AWS Aurora.

The problem is not as simple as replacing a relational solution, such as SQL, with a NoSQL alternative. Both [36, 37] show that a number of architectural and performance considerations must be taken into account to select within one of these categories, even after the decision to use either option is finalized.

4.2 RQ1 – DBMS candidates

To answer the first research question, a secondary literature review was conducted. The selection process of this review resulted in 29 sources [11, 31, 38–64] to extract information from. The frequency of appearance of the different database management systems encountered in the literature review is presented in Figure 4.1. For legibility reasons, the labeled databases in the figure were featured in at least two sources while the 'others' category contains the databases that were only featured in one source. This category encompasses the following database management systems: IBM DB2, MS Access, Microsoft SQL Server, MemSQL, Spark, Memcached, Firebase, MonetDB, Neo4j CALL, ArangoDB, Redis, RethinkDB, Riak, OrientDB, ScyllaDB, NuoDB, CockroachDB, VoltDB, TIBCO Active Spaces, MySQL NDB Cluster, pgvector, Weaviate, Milvus, Qdrant, Chroma, and Polyglot. The most frequently featured database in the sources was MongoDB, featured 15 times, followed by MySQL, featured 14 times, and PostgreSQL/Neo4j, both featured 6 times each.

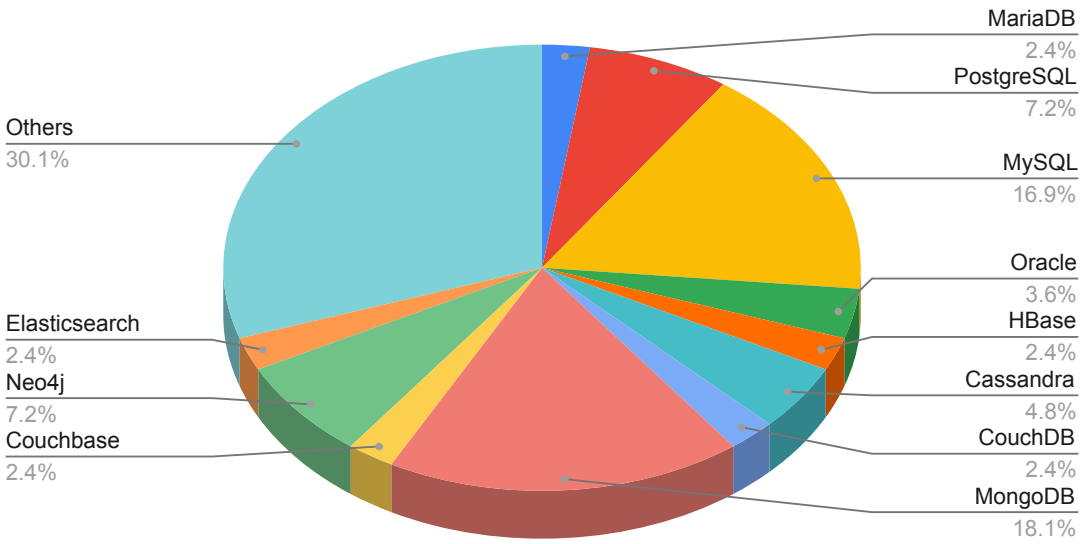


Figure 4.1: Pie chart showing frequency of appearance of each DBMS in literature review 2.

Through the process described in the end of subsection 3.4.1, the different DBMSs were awarded points based on how many peers they outperformed per test. The results are presented in Figure 4.2 and Figure 4.3, where the latter shows the databases'

points normalized by the number of sources they were featured in, to reduce the advantage of being featured in many reports. This sort of normalization instead favors the databases featured in few sources with many tests, such as Redis and the vector databases.

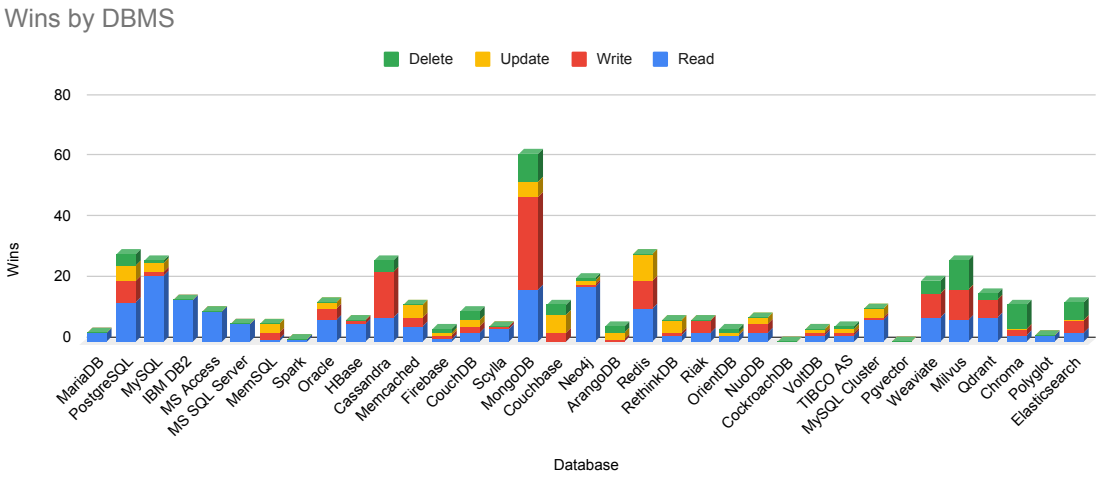


Figure 4.2: Stacked column chart showing total wins by DBMS per category in literature review 2.

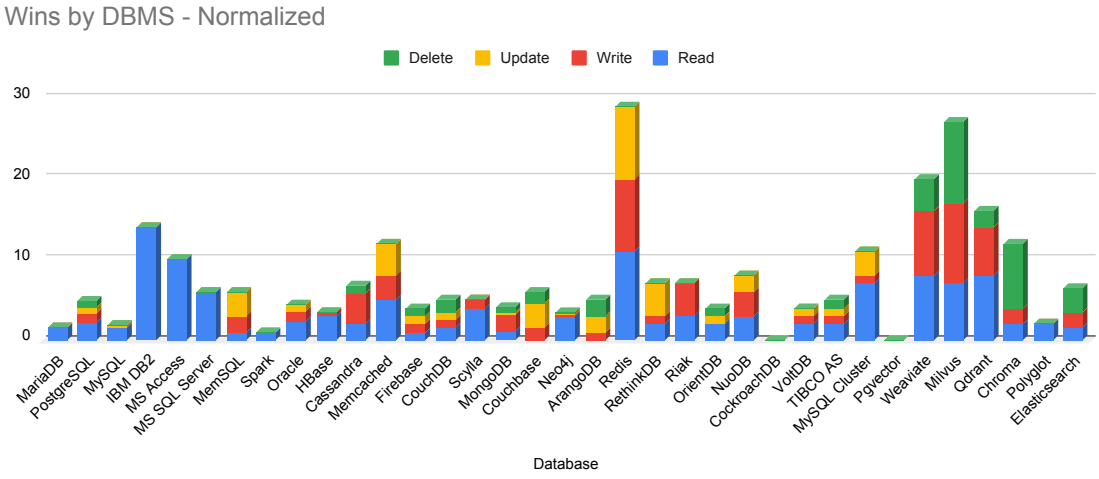


Figure 4.3: Stacked column chart showing normalized (divided by number of sources featured in) total wins by DBMS per category.

4.3 RQ2 – Selection framework

4.3.1 Artifact design

The final iteration of the DBMS framework features all the expected errors and drawbacks of the frozen literature review methodology. After the final iteration, the result has 6 decisions, 3 terminators, 8 outputs in the form of database options, as seen in Figure 4.4. Together these cover 23 possible use cases for the user. It is clear to see that even though effort was made to organize the flowchart components in a legible manner, such an approach is up to debate in terms of scalability.

The undetermined sections, as well as the previously pragmatic approach can now be held up in contrast to results in subsection 3.5.1. It is here plain to see that built-in vector support is generally the better choice regardless of how much vector data is handled.

4.3.2 Expert survey

In evaluating the usefulness of the recommendation framework, a brief survey was conducted. The survey yielded three responses, which provide valuable insights from industry experts. The responses are presented in Table A.1.

The Q1 responses suggested that the recommendation flowchart was overly simple and failed to take many practical factors into consideration and did not feature enough options. It was also suggested that the nature of a decision tree like this is too black and white and lacks the nuance that a points-based selection could offer. One survey respondent pointed out the many non-performance related omissions. While this contrasting view was legitimate, it also was not relevant to the problem statement.

The responses to Q2 mostly conveyed that the recommendation framework had some potential but was limited by the small assortment of databases, with the exception of a brief, and complete rejection by one respondent.

Responses to Q3 show that a majority of respondents understood the design language and logical flow of the recommendation framework as well as the way it allows the combination of vector and non-vector databases to be selected. The outlier expressed frustration with the vector or non-vector dichotomy, but provided little elaboration on this.

Q4 gave the respondents a chance to weigh in on any databases they consider to be missing from the recommendation flowchart. One respondent considered Oracle Database and PostgreSQL to be missing. A second respondent noted that missing options are likely, but did not give examples. The final respondent referenced back to an earlier response, in which no recommendations toward addition or removal were made.

The last question, Q5, served as a catch-all question at the end of the survey. The responses did not contain much that had not already been discussed in the earlier questions.

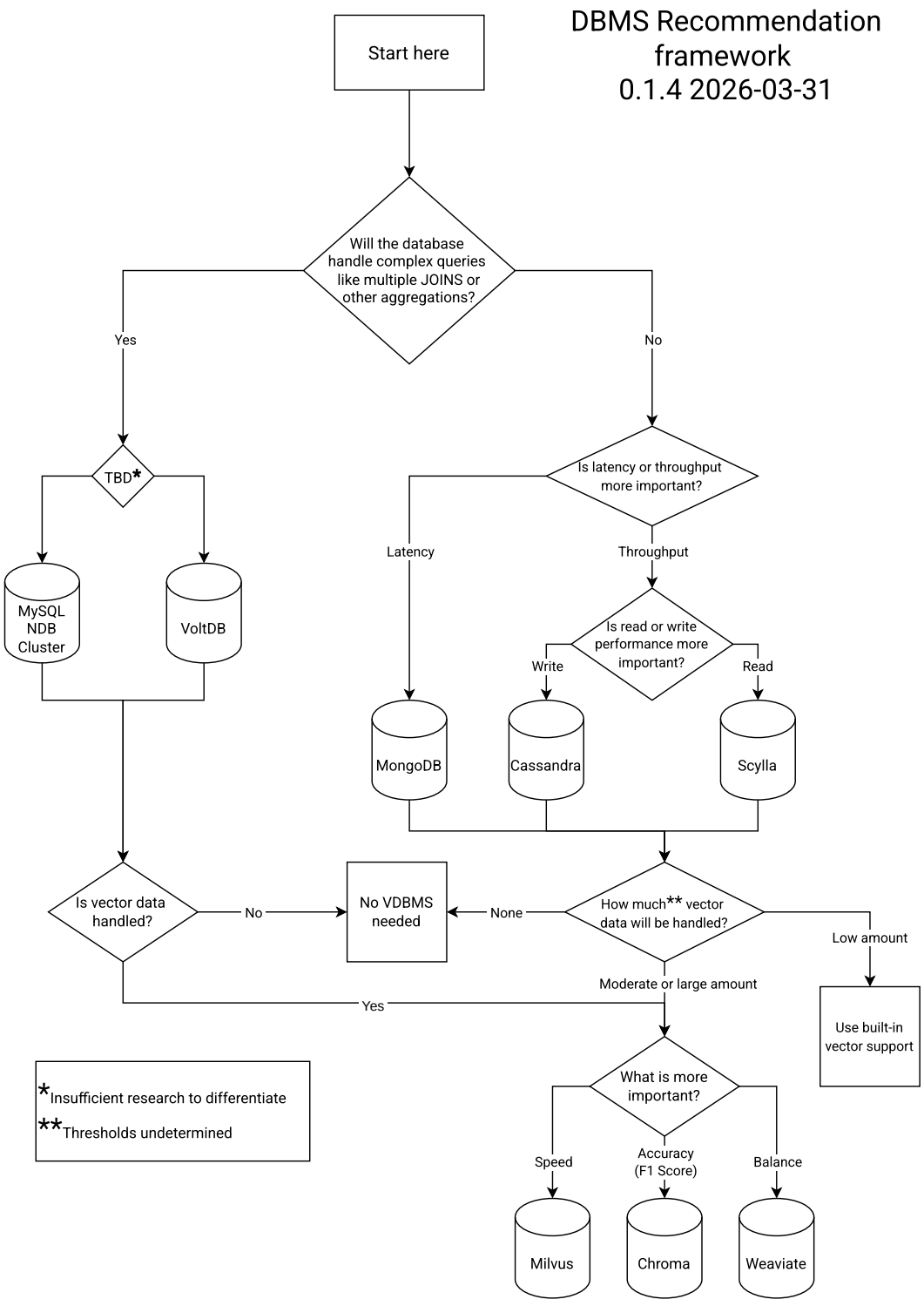


Figure 4.4: DBMS recommendation framework

4.4 RQ3 – Performance comparison

The quasi-experiment in the form of a series of benchmarks was performed to compare the relative performance of the chosen DBMS solutions featured in the recommendation framework.

During the benchmarks, the performance metrics throughput and latency, as well as resource usage in terms of CPU and memory usage were measured and are presented in the figures below.

The first two throughput graphs feature the write-heavy workloads. In these graphs, most solutions followed the expected ramping curve until the traffic load became too much, and different database solutions started deviating from the requested throughput. In Figure 4.5, the Cassandra standalone solution has the highest and most stable throughput, with Scylla, MongoDB and MySQL becoming increasingly irregular. At the bottom of the graph, we find the Cassandra-Milvus and Mongo-Milvus solutions, which were unable keep up with the requested throughput past the first few seconds and only reached approximately 450 and 180 operations per second respectively. Figure 4.6 displays familiar, yet slightly different results. The overall throughput was marginally lower than in the previous graph. The Scylla standalone solution achieved the highest throughput, followed by Cassandra standalone, Scylla-Milvus, and MySQL-Milvus, in that order. The variations are smaller than in the previous graph, most likely because of the lower amount of vector data. The last two throughput graphs – Figure 4.7 and Figure 4.8 – show that the throughput in the read-heavy benchmarks did not follow the ramping curve for more than a few seconds before being overwhelmed by the traffic. In Figure 4.7, the throughput of Cassandra-Milvus, Cassandra-standalone, and MySQL-Milvus increased rapidly before declining and rising again. Scylla-standalone, Scylla-Milvus, and Mongo-Milvus managed a lower, but more stable throughput. Figure 4.8 shows a slightly higher throughput overall, with MySQL-Milvus and Cassandra-standalone performing strongly initially, but later showing a decline.

Herein the null-hypothesis presented previously in subsection 3.5.1 may be disproved. The results showed differences in which performance trivially doubles, or in extreme cases of latency, showed a difference greater than 2 orders of magnitude. Due to such immense differences, it was considered that calculating any reasonable level of statistical deviation would serve no purpose, i.e, the disproval of the null-hypothesis was considered self evident.

For all tested workloads, the standalone solutions radically outperformed the other solutions as seen in Figure 4.10 and Figure 4.12. This gap was marginally affected in Figure 4.9 and Figure 4.11 the vector workload increases, arguably in favor of the dual-database solution, although this solution regardless performed poorly.

For none of the tested databases was there a notable increase in latency over time, even as the ramping curve increased demand. The shown values in each of the graphs are an average for any given second. For this reason, the jagged look of the dual-database solutions suggested periods of either consistently poor performance, or a singularly extreme outlier. In contrast, the standalone solutions appeared relatively stable. The clear exception to this is seen in the vector heavy workloads showcased in Figure 4.11 and Figure 4.9. Towards the end of these tests, both standalone solutions were observed to wildly fluctuate. This coincided roughly with the database

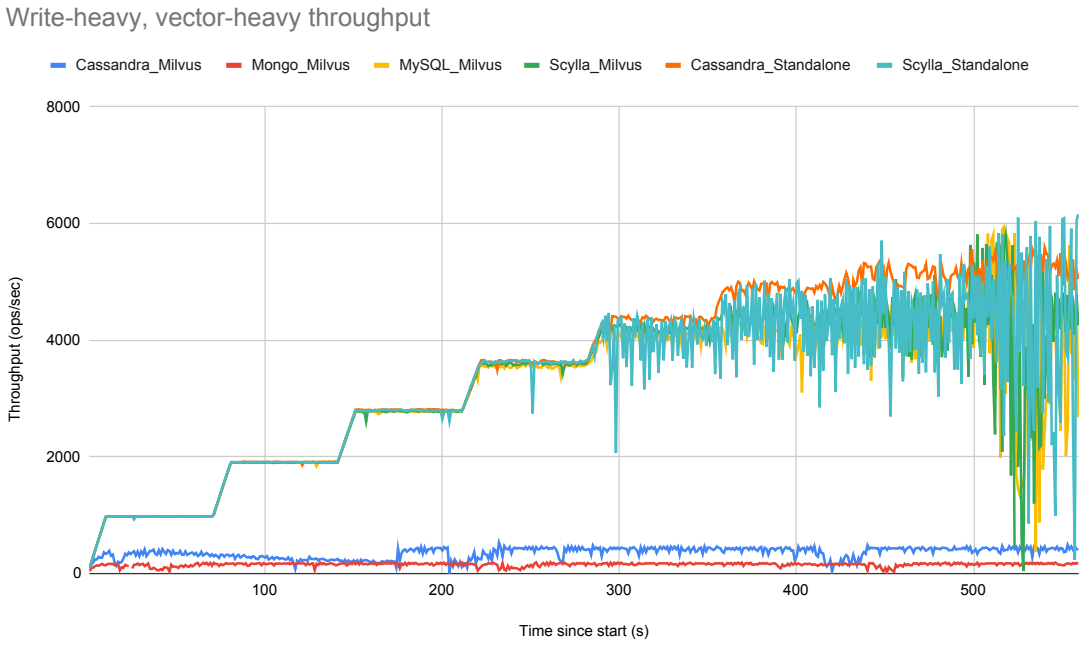


Figure 4.5: Graph of throughput over time for a write-heavy, vector-heavy traffic pattern.

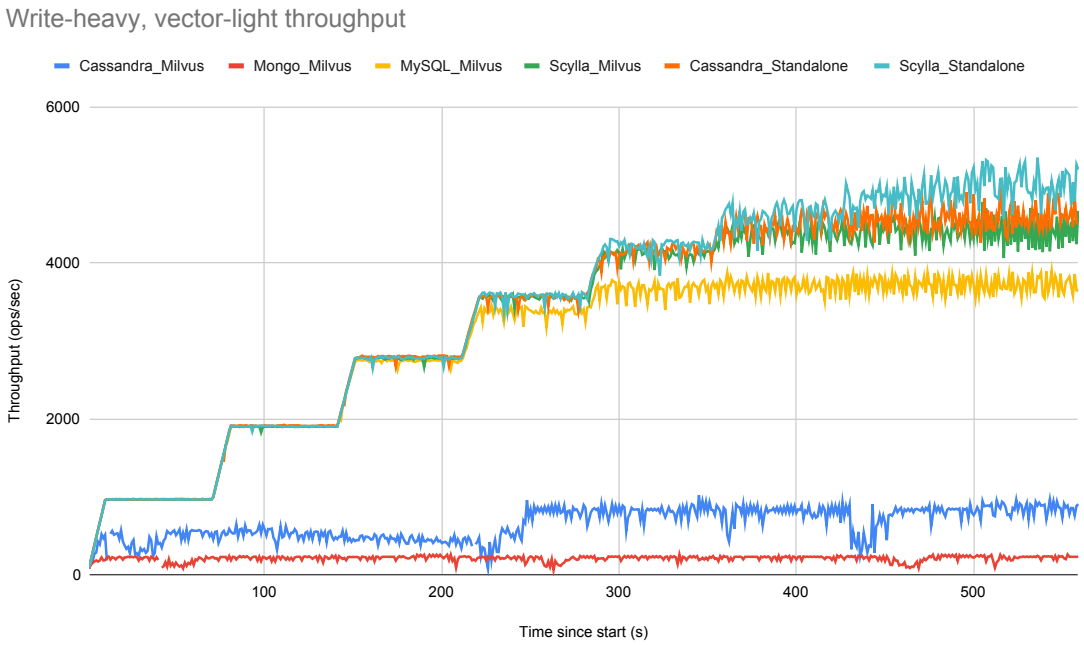


Figure 4.6: Graph of throughput over time for a write-heavy, vector-light traffic pattern.

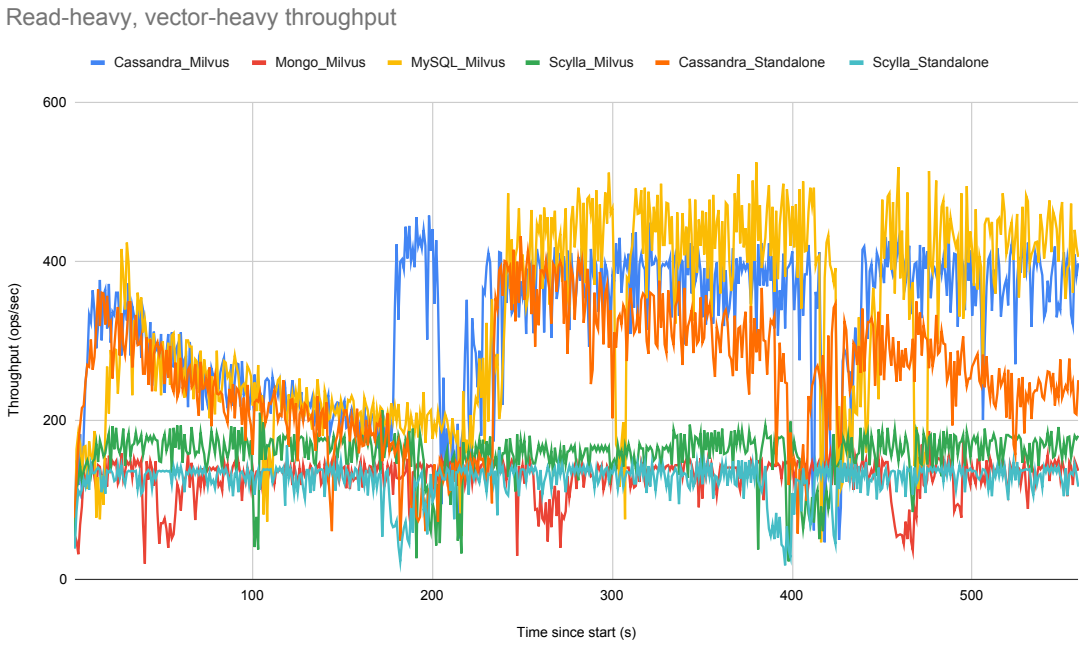


Figure 4.7: Graph of throughput over time for a read-heavy, vector-heavy traffic pattern.

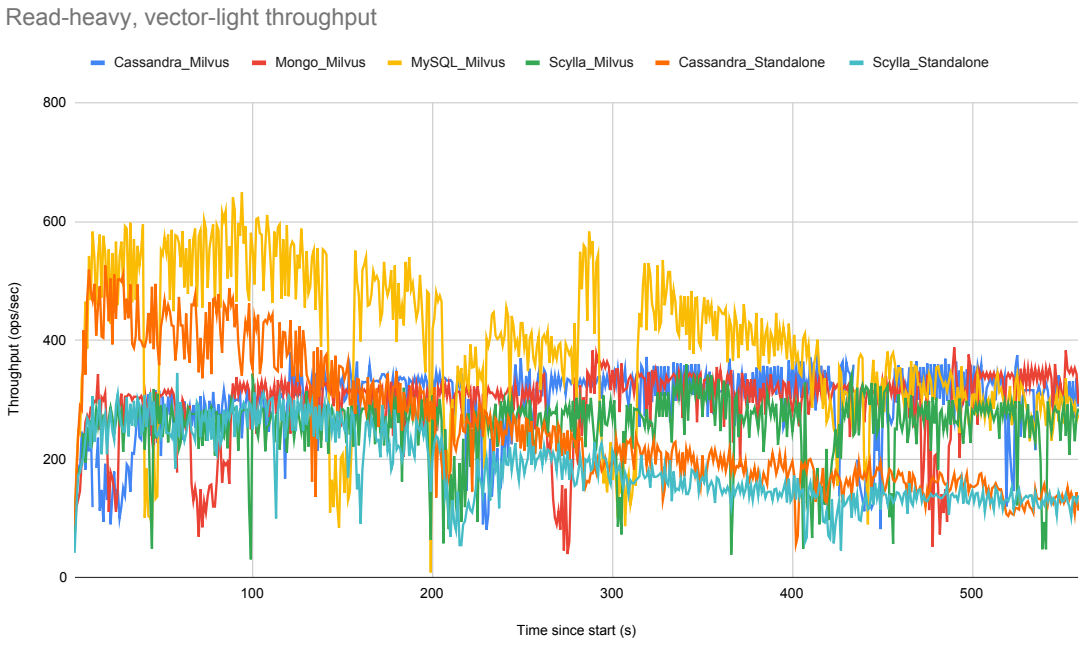


Figure 4.8: Graph of throughput over time for a read-heavy, vector-light traffic pattern.

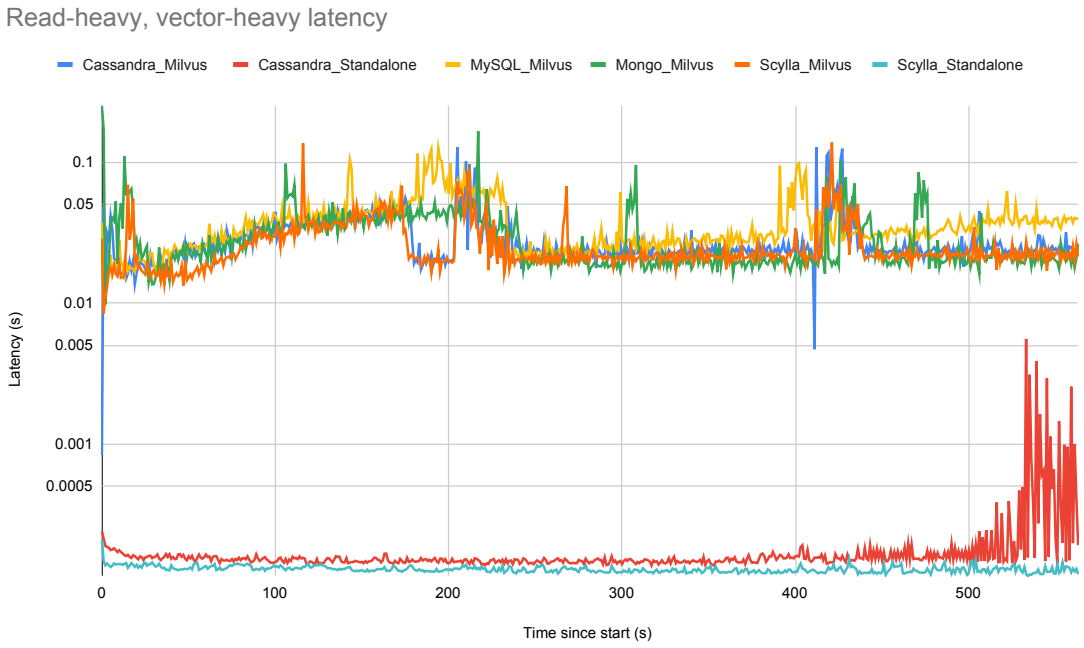


Figure 4.9: Logarithmic graph of latency over time for a read-heavy, vector-heavy traffic pattern.

throughput threshold being reached.

On the matter of resource utilization, some of the the dual-database solutions hovered far lower than expected. MongoDB and Scylla hardly utilized provided memory when forced to share space with Milvus. The remaining solutions made use of the memory in a steady fashion, many of them preemptively allocating a large chunk, as seen in Figure 4.14 and Figure 4.13. Standalone Scylla was the outlier in memory consumption, starting at low usage and steadily growing alongside the ramping curve until the very end of the test. In both vector heavy and light workloads, this growth would approach, but never surpass that of the standalone Cassandra solution.

The CPU results seen in Figure 4.15 and Figure 4.16 showed the recurring failure on the part of the combined solutions to utilize allocated resources, at least in the case of the solutions also tested standalone. MongoDB and MySQL appeared not to run into this issue as severely. Between the standalone solutions Scylla consistently showed lower CPU utilization than Cassandra, which had the highest usage of any tested workload.

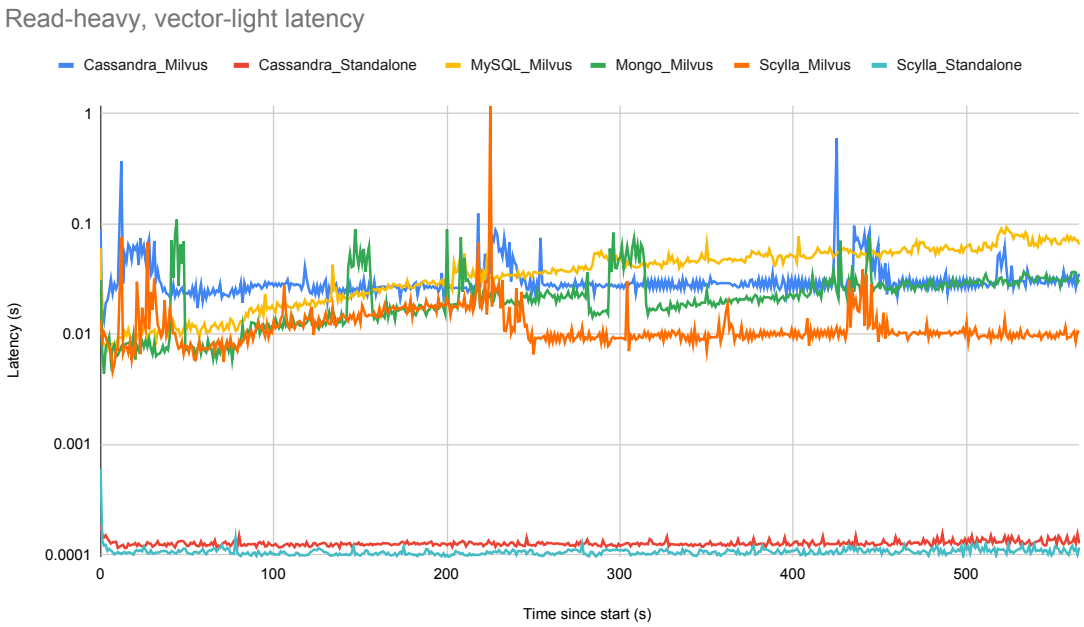


Figure 4.10: Logarithmic graph of latency over time for a read-heavy, vector-light traffic pattern.

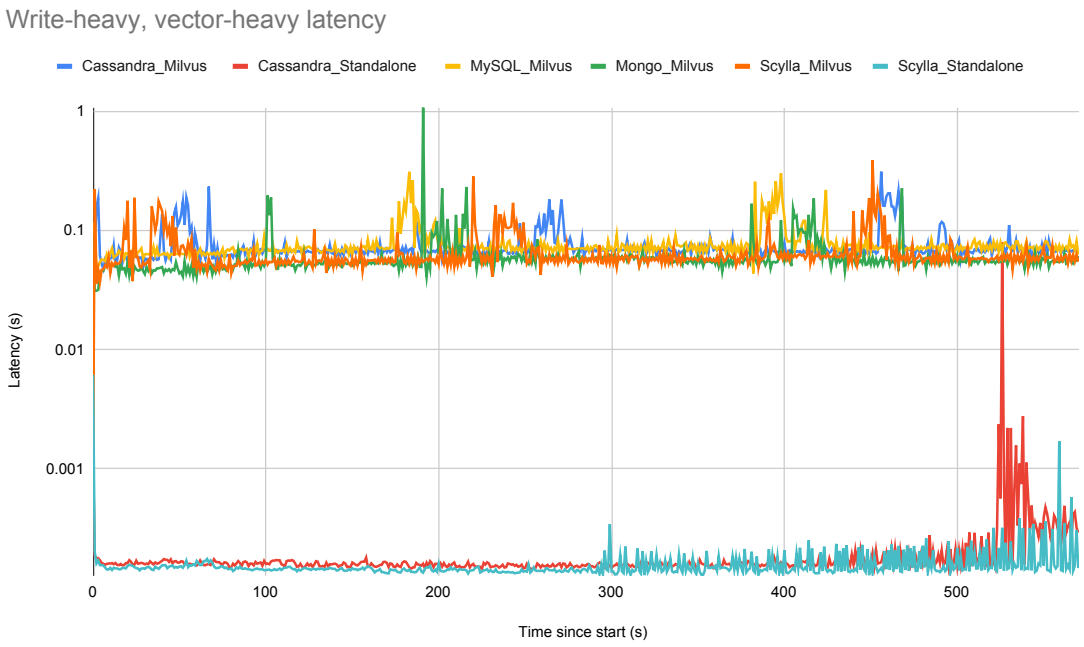


Figure 4.11: Logarithmic graph of latency over time for a write-heavy, vector-heavy traffic pattern.

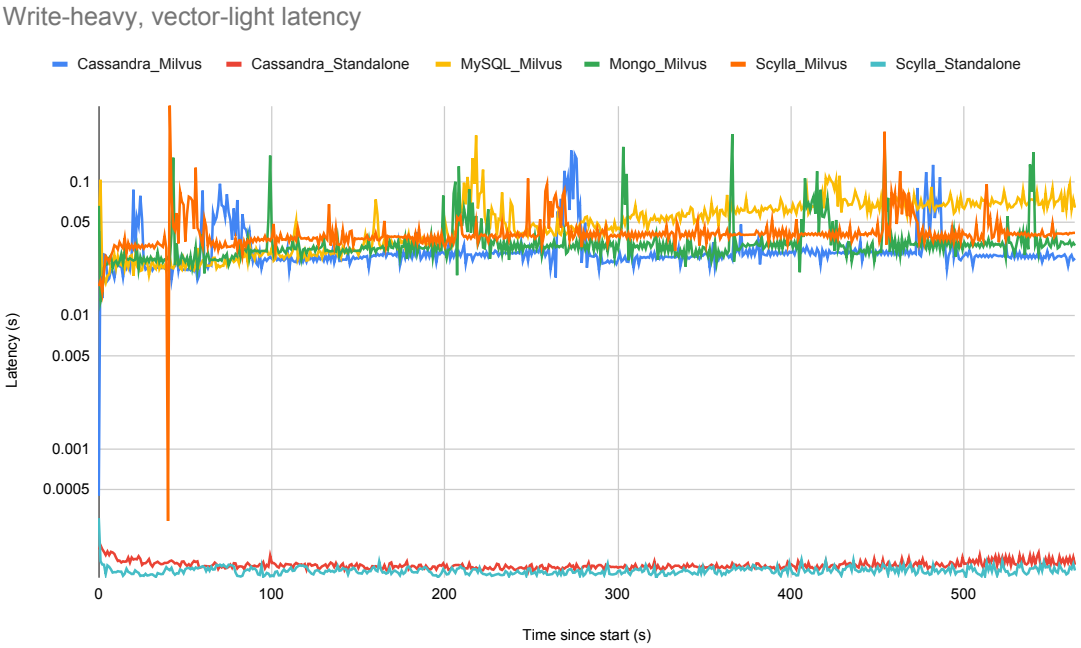


Figure 4.12: Logarithmic graph of latency over time for a write-heavy, vector-light traffic pattern.

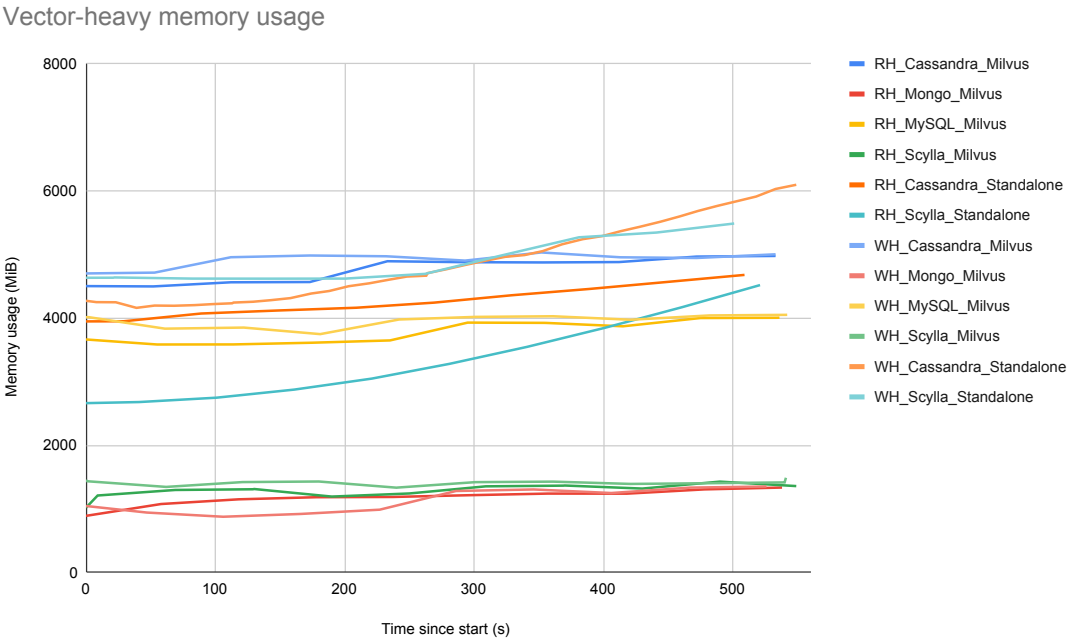


Figure 4.13: Graph of memory utilization over time for a vector-heavy and read/write-heavy traffic pattern.

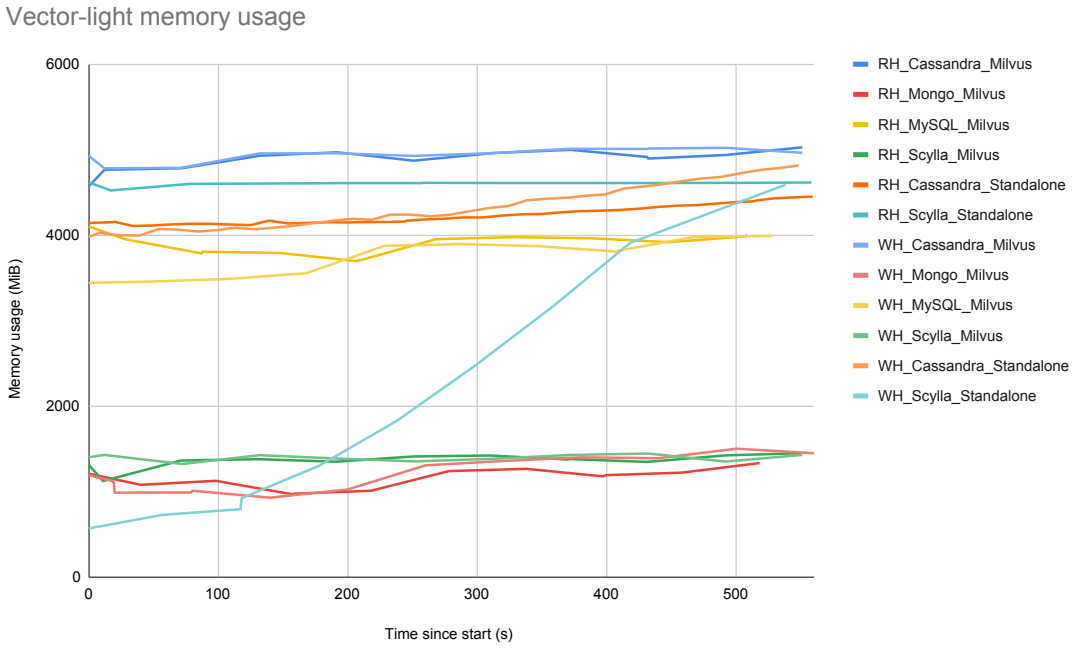


Figure 4.14: Graph of memory utilization over time for a vector-light and read/write-heavy traffic pattern.

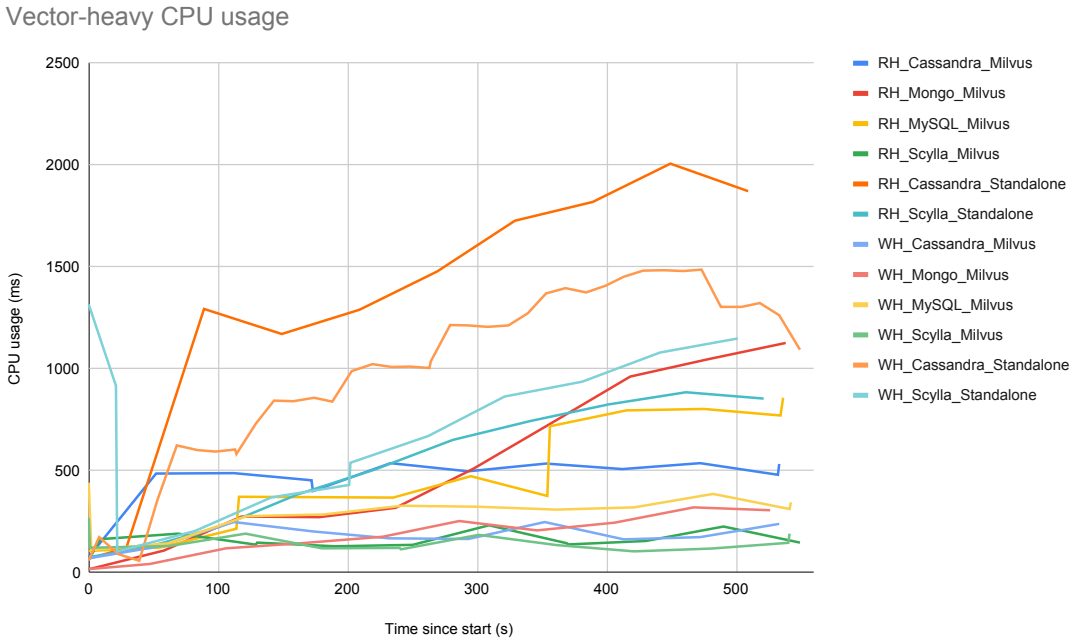


Figure 4.15: Graph of CPU utilization over time for a vector-heavy and read/write-heavy traffic pattern.

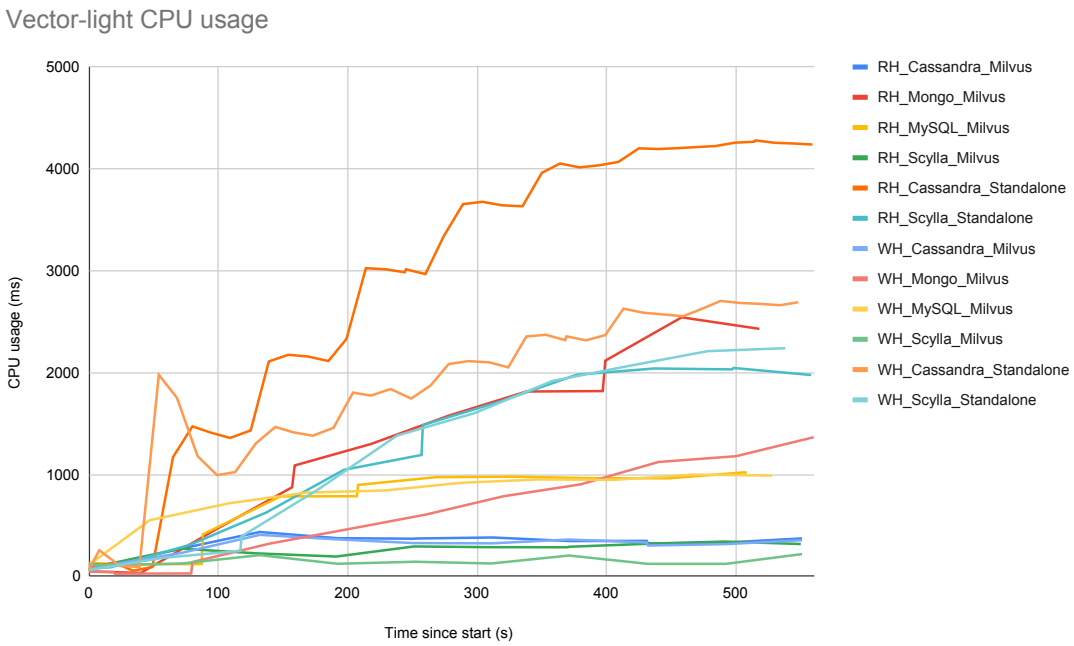


Figure 4.16: Graph of CPU utilization over time for a vector-light and read/write-heavy traffic pattern.

Chapter 5

Discussion

5.1 Experiment results

With respect to the results, the question of which databases out of the investigated options prevails. The second literature review identified a set of databases for final inclusion: MySQL Cluster, Cassandra, Scylla, Milvus, and MongoDB. Although VoltDB appeared in the evaluation framework, it was excluded for the reasons previously outlined. The survey also prompted consideration of PostgreSQL; however, its extended configuration was omitted from the study owing to inadequate performance on vector workloads.

Removal of MongoDB warranted further consideration. Its vector-search capabilities were not available in the standard open-source distribution, and the performance of the combined solution proved unsatisfactory, raising doubts about the viability of its inclusion.

Consequently, the final set of candidate systems comprised three databases: MySQL Cluster and Milvus, Cassandra, and Scylla.

This final selection was based upon the performance of these options in terms of throughput. The inferior latency of the MySQL Cluster and Milvus configuration may have been the result of MySQL Cluster being a NewSQL DBMS; which would align well with the results observed by most sources in the first literature review [43, 45, 48, 49, 51, 52, 56].

Alternatively, this result could be caused by an inferior latency from Milvus. The low resources provided to the vector DBMS could very well have resulted in poor performance. While a comparison between these results and those observed by [11] is fallacious [20], their observed results leave no indication that Milvus should be the cause. It can thus initially be concluded that the latency observed in MySQL Cluster was expected, and as such its inclusion among the final 3 was warranted.

Scylla and Cassandra both showed impressive and similar performance in terms of latency when in their standalone configurations. When configured in combination with Milvus, however, these databases also performed similarly to MySQL cluster. This result heavily implied Milvus was the cause of the increased latency values. It is worth noting that Milvus expects a far greater amount of resources than was given. It is plausible that the observed latency gap may be reduced, or eliminated, assuming all databases are given the resources they require.

The dual-database solutions did not universally perform poorly in terms of throughput however, where they commonly showed similar results to standalone solutions, showing that while not ideal, such a solution is indeed workable.

Though the discussion of binary inclusion can be concluded, there remains the matter of relative performance between the final selection. The conducted quasi-experiment shows a relatively clear picture of the relative database performance. The top options correlated with those identified as "best", where a "good" performer showed high throughput at low hardware utilization. In this respect, the hardware utilization charts are considered incomplete. It would have been ideal to run each test for a longer period, to see the development at a longer timescale.

The memory usage graphs can each be divided into 2 groups. Among databases with high memory usage were Cassandra in both its configurations and MySQL Cluster, alongside the read-heavy standalone Scylla result. Near the bottom, with generally lower utilization were the MongoDB results and the the remaining Scylla charts, with one exception of note; the read-heavy standalone Scylla test, which showed a comparatively anomalous climb over time in terms of memory usage. While somewhat stalling near the top, it is equally plausible for this utilization to keep climbing as it is for it to settle in among the others. This result implies a number of things. Firstly, the data read implementation found in Scylla does not claim memory it will not utilize. Secondly, once the demand for read operations hikes, so does the memory usage. Finally, while the same effect can be observed in the vector-heavy workloads, it is not at all as pronounced, implying that the vector operations are more expensive as a baseline.

Another matter of note in the vector heavy workloads, and to a lesser extent the vector light workloads is that as Scylla's memory usage increases, eventually so does Cassandras for the same operation types. While in vector-light traffic, these options effectively come close enough to be withing margin of error, such is not the case in vector-heavy workloads. For those workloads, it appears that Cassandra's memory usage eventually grows at a pace similar to that of Scylla, although with a higher total consumption.

MySQL Cluster shows a similar usage profile in terms of memory to Cassandra, however marginally lower almost across the board. It also does not show much of a tendency to grow as the other top contenders.

Finally the results for MongoDB are impressive at the surface-level. Utilizing very few system resources generally. It should however in greater context be more appropriately described as failing to utilize these resources, as it does not translate this low resource consumption into either throughput nor impressive latency.

It is tempting to assume some form of error in the setup of MongoDB, however, just as the other databases MongoDB was given minimal configuration. Thusly, the poor performance could be due to numerous factors. As the only sharded database in the tested cohort, it may not be suited for the specific workload and environment.

Regarding CPU utilization, most databases show a result which correlates more with the ramping curve.

Among the top 3 options, Cassandra is outperformed in CPU utilization from both alternatives. How well this scales in the long-term remains to be determined. Cassandra in the standalone configuration at any workload shows the highest CPU utilization across the board. It can be speculated that this increased usage is the direct result of the heavier memory-safe Java Programming Language, as this is a considerable distinction between it and the notably better performing C++ rewrite found in Scylla; all other things being equal.

MySQL Cluster shows even better scaling, with a nearly flat profile in most of its CPU utilization results. This is inline with the other dual database solutions, where the reduced CPU usage can generally be attributed to the reduction in allocated resources. The exception to this is the MongoDB solution which shows a similar profile to Scylla standalone in its read heavy workload.

As with prior question, the quasi-experiment also provides the clearest picture with regard to the general architecture of the DBMS solutions, those being dedicated Going through each workload in order, starting with Figure 4.5 and Figure 4.6. These graphs can generally be divided into 2 cohorts; high and low performers.

The well-performing in terms of write throughput in either workload are MySQL, both standalone solutions, and the combined Scylla solutions. This shows a relatively even split in terms of whether or not a dual solution performs well in write performance, while the standalone solutions semi-consistently perform well.

In vector-heavy writes, general flakiness in throughput can be observed both in dual and standalone solutions, though Cassandra standalone appears immune to this effect. This is a conclusion-wise unsatisfying answer, as it points towards a "your results may vary" type of answer in which the database itself matters as much of not more than the architectural setup. Regardless, although dual solutions often end up near the top of the charts in these tests, the risk of performing as dual Cassandra or MongoDB appears a very real risk, and as such a standalone solution appears here a safe and rational choice.

In moving on to read-heavy performance in Figure 4.7 and Figure 4.8, we instead observe a much more even split, with the Scylla standalone solution consistently remaining near the bottom of the chart. This could very well be due to a poor vector read implementation, however its results do not appear to improve by much as the vector data drops in quantity. In fact, in vector-light, read-heavy workloads Cassandra also struggles, while the dual-solutions dominate.

This is especially pronounced in the vector-heavy throughput. This may be attributed to Milvus. It can quite safely be assumed that its vector search implementation is superior enough to the standalone solutions to where it is allowed to shine in this workload, which is catered specifically to its use case.

The idea of dual database deployments, one with and one without vector capability appears mostly to be a poor one. In terms of latency, these solutions were heavily outperformed by databases with native support, although MySQL Cluster manages occasionally impressive bursts in throughput capacity. The scenario in which this is not the case is in very heavy vector search applications, where throughput is a greater priority than latency.

On the other hand, standalone solutions appear generally both more consistent and performant. While not universally true, the case against these integrated solutions appears narrow.

On a further note, the lack of accuracy testing for the vector portion of the data does bring with it potential issues. While Milvus certainly may have outperformed the standalone solutions, this says nothing of the correctness of the output, and thus it is entirely plausible that the performant option regardless is of little to no value.

Generally, the experiment yielded unexpected results. As per [11], it was predicted that hybrid database approaches such as pgvector would be outperformed by a dedicated vector database solution. Although the results are inconclusive due

to the reduced specifications allocated to the cluster, one may regardless come to the conclusion that vector data is best handled natively by all-encompassing hybrid database solutions in all but the most vector-heavy cases.

5.2 Systematic selection of DBMSs

The final outcome of the second literature review yielded the recommendation framework. This was a partially successful first approach to the problem, though not without an array of flaws. Although certain factors indicate the approach must be entirely altered, others indicate the flowchart approach is flawed only in its current iteration, and may be refined. The evaluations, both the expert survey and quasi-experiment inform which changes would be suitable for future iterations, and which alternate approaches more befit this problem statement. While this section will discuss what changes ought to be made in a potential future iteration, the creation of such a refinement is considered out of scope for this thesis, as has been specified by omission in section 3.2.

5.2.1 Expert survey

The respondents to the survey were in agreement on a flowchart not capturing the act of weighing ones options. A preference is indeed ordinal rather than binary, thus going down one path in a flowchart results in the user feeling as if the option left behind will be entirely neglected; even though this is not entirely the case. For example, when selecting between latency and throughput, a user may assume that choice of the latter will result will recommend a database capable of a massive quantity of simultaneous operations, but an egregious latency – whatever such a term means to the user. This echoes the sentiment expressed by [4]. Though the framework was designed with the intention of displaying the lack of "one size fits all" solutions, the binary nature of a flowchart junction turns out to work against this idea.

Additionally, the flowchart approach hands at once much and little control to its reader. Should a reader disregard certain iconography, or perhaps find a junction at which they are unwilling to compromise on, there is no method of ignoring said junction and proceeding regardless. This may be remedied with a third option to skip, or to compromise on the given alternatives. This increased complexity, alongside the idea of increasing the number of factors considered beyond performance appears intuitively to rapidly reduce the legibility of the framework.

A better approach considered is that of a quiz-like structure. Each question in such a quiz alters differing backend values corresponding to database preference without immediate exclusion. Such an approach would give way to a greater degree of weighing alternatives. A compromise in such a quiz would simply result in neither weighed database option being "given points", effectively skipping the question. The proposed system would be of similarity to those proposed by [22], though much further simplified.

5.2.2 Experiment impact on DBMS selection

The experiment showed dual-database solutions to be insufficient in all but the most read and vector intensive workloads. Although all tested solutions struggled in data, the standalone solutions managed to perform in write-heavy workloads regardless, while performing similarly to the dual solutions in reads. This suggests that the many vector specific optimizations performed by a vector-exclusive database such as Milvus outweigh the the performance cost imparted by reduced resource availability, even if only in the most favorable circumstance. It appears then, that the assumption of using a separate vector-database under heavy vector operations was in fact a correct one, although the framework should be tweaked to exclude the moderate case.

Considering the near equal results when vector read operations make up 25% of the data versus the occasionally doubled performance seen during 75%, it can be assumed that a likely middle-ground is found for a 50% vector read workload. Such a middle-ground would likely result in an occasional 50% performance uplift in the best case scenario. Thusly, a dual-database approach is not suitable for the moderate vector data use case. Of course, this conclusion runs the risk of being a form of middle-ground fallacy, as the relation between vector data ratio and real throughput may not be linear.

Certain database solutions also featured in the recommendation framework could not be in turn featured in the experiment. VoltDB for instance has since the time of the research papers used in Literature review 2 closed sourced its code-base, and now exists in the form of Volt Active Data[1]. The remaining open-source projects appear to be generally unmaintained, and cannot thus be fairly compared to the other options. A similar case is true for MongoDB. The methodology for this thesis required for each database solution to be locally deployable. Native vector support exists under MongoDB only in so far as the user is willing to use cloud solution[2]. This erased a standalone MongoDB test from the benchmark, while keeping it eligible for the framework – which necessarily based its form purely on literature review, and thus could not take unmentioned feature details into account.

[1] `https://www.voltactivedata.com/`

[2] `https://www.mongodb.com/products/platform/atlas-database`

Chapter 6

Conclusions and future work

6.1 Conclusions

This thesis presented 3 research questions as its aim. The database solutions to best support the mixed workloads are decisively the standalone Cassandra and Scylla solutions in write-heavy workloads, and arguably the dual database Scylla, Cassandra or MySQL solutions in the read-heavy vector-heavy workloads.

Comparing the database solutions, it is clear that standalone Cassandra and Scylla outperform the others in terms of latency in all tested workloads. They do this while using equal or greater amounts of resource utilization. In relation to one another, Scylla generally had lower hardware utilization, for the same workload, but this showed a tendency of rising over time. It may be noted this result is exclusive to the specific test-bed.

The impacting metrics are partially answered by this thesis. The literature review shows extended approaches are inferior to native ones. The experiment shows dedicated databases are superior to hybrid dual-database solutions approaches.

The proposed flowchart structure is a bare-bones, minimal approach to systematic DBMS solution selection. It seemingly solves a narrow subset of a wide problem. After thorough evaluation however, the framework ought to undergo sweeping large changes, such as a change of format, to a points-based approach. The scope should also be widened to include a wide variety of non-performance characteristics, and a greater selection of databases.

6.2 Future work

Through the work described in this thesis, an initial contribution has been made to the field of mixed vector workload testing. It has done so via a the building of a rudimentary benchmarking tool, inference of results based upon prior research, and an experiment determining the validity of certain such inferences. There exist a multitude of further research paths within this area, most critically the addition of further nuance to the tests. Specifically, the distribution of resources in dual solutions can be tuned in accordance with expected workload, or an additional 50% split to the read/write, vector light/heavy data variations. This, alongside a test with more available resources for each database should additionally provide conclusions which may even disprove the results found in this thesis.

The recommendation framework can be further improved, either by expanding the number of branches provided to a user, or by altering the structure to that of a

quiz rather than a flowchart. It is clear from the survey results that a larger scope solution, such as demonstrated by Brahimi [22], although perhaps less intuitive and therefore usable, regardless, proves more useful due to a more encompassing scope.

References

[1] M. T. Özsu and P. Valduriez, *Principles of Distributed Database Systems*. Cham: Springer International Publishing, 2020.

[2] Z. Jing, Y. Su, and Y. Han, "When large language models meet vector databases: A survey," p. 7–13, Feb. 2025.

[3] J. J. Pan, J. Wang, and G. Li, "Survey of vector database management systems," *The VLDB Journal*, vol. 33, p. 1591–1615, July 2024.

[4] M. Stonebraker and U. Cetintemel, ""one size fits all": an idea whose time has come and gone," in *21st International Conference on Data Engineering (ICDE'05)*, pp. 2–11, 2005.

[5] B. Ingram, "How discord stores trillions of messages," Mar. 2023.

[6] S. Harizopoulos, V. Liang, D. J. Abadi, and S. Madden, "Performance tradeoffs in read-optimized databases," in *Proceedings of the 32nd International Conference on Very Large Data Bases*, VLDB '06, p. 487–498, VLDB Endowment, 2006.

[7] M. Wiesmann, F. Pedone, A. Schiper, B. Kemme, and G. Alonso, "Understanding replication in databases and distributed systems," in *Proceedings 20th IEEE International Conference on Distributed Computing Systems*, pp. 464–474, 2000.

[8] M. Ahamad and M. Ammar, "Performance characterization of quorum-consensus algorithms for replicated data," *IEEE Transactions on Software Engineering*, vol. 15, no. 4, pp. 492–496, 1989.

[9] M. Hassan, "Big data, big decisions choosing the right database," 2024.

[10] W. Khan, T. Kumar, C. Zhang, K. Raj, A. M. Roy, and B. Luo, "Sql and nosql database software architecture performance analysis and assessments—a systematic literature review," *Big Data and Cognitive Computing*, vol. 7, no. 2, 2023.

[11] T. Sakal Franciškoviċ, N. Todoroviċ, J. Hrnjak, and V. Dimitrieski, "Vector database benchmarking: A face retrieval use case," *Lecture Notes in Computer Science*, vol. 16043 LNCS, pp. 28 – 43, 2026.

[12] M. Wiesmann and A. Schiper, "Comparison of database replication techniques based on total order broadcast," *IEEE Transactions on Knowledge and Data Engineering*, vol. 17, no. 4, pp. 551–566, 2005.

[13] J. Wang, X. Yi, R. Guo, H. Jin, P. Xu, S. Li, X. Wang, X. Guo, C. Li, X. Xu, K. Yu, Y. Yuan, Y. Zou, J. Long, Y. Cai, Z. Li, Z. Zhang, Y. Mo, J. Gu, R. Jiang,

Y. Wei, and C. Xie, "Milvus: A purpose-built vector data management system," in *Proceedings of the 2021 International Conference on Management of Data*, SIGMOD '21, (New York, NY, USA), p. 2614–2627, Association for Computing Machinery, 2021.

[14] M. Nicola and M. Jarke, "Performance modeling of distributed and replicated databases," *IEEE Transactions on Knowledge and Data Engineering*, vol. 12, no. 4, pp. 645–672, 2000.

[15] M. Stonebraker, C. Bear, M. Cherniack, T. Ge, N. Hachem, S. Harizopoulos, J. Lifter, J. Rogers, and S. Zdonik, "One size fits all? - part 2: Benchmarking results," *CIDR 2007 - 3rd Biennial Conference on Innovative Data Systems Research*, Jan. 2007.

[16] H. Boral and D. J. DeWitt, "A methodology for database system performance evaluation," *SIGMOD Rec.*, vol. 14, p. 176–185, June 1984.

[17] A. Ohlsson and M. Persson, "A comparison in performance between a selection of databases," 2019.

[18] T. Taipalus, "Vector database management systems: Fundamental concepts, use-cases, and current challenges," *Cognitive Systems Research*, vol. 85, p. 101216, 2024.

[19] T. Taipalus, "Database management system performance comparisons: A systematic literature review," *Journal of Systems and Software*, vol. 208, p. 111872, 2024.

[20] M. Raasveldt, P. Holanda, T. Gubner, and H. Mühleisen, "Fair benchmarking considered difficult: Common pitfalls in database performance testing," *Proceedings of the Workshop on Testing Database Systems, DBTest 2018*, 2018.

[21] F. Zahedi, "Applications and implementation data-base management system evaluation and selection decisions," *Decision sciences*, vol. 16, no. 1, pp. 91–116, 1985.

[22] L. Brahimi, L. Bellatreche, and Y. Ouhammou, "A recommender system for dbms selection based on a test data repository," in *Advances in Databases and Information Systems*, (Cham), pp. 166–180, Springer International Publishing, 2016.

[23] A. Mateen, B. Raza, M. M. Awais, and M. Sher, "Selection of the best dbms: A fuzzy based multi-objective decision making approach," in *Proceedings of the International MultiConference of Engineers and Computer Scientists*, vol. 1, 2011.

[24] C. Wohlin and P. Runeson, "Guiding the selection of research methodology in industry–academia collaboration in software engineering," *Information and Software Technology*, vol. 140, p. 106678, 2021.

[25] P. Offermann, O. Levina, M. Schönherr, and U. Bub, "Outline of a design science research process," in *Proceedings of the 4th International Conference on Design Science Research in Information Systems and Technology*, DESRIST '09, (New York, NY, USA), Association for Computing Machinery, 2009.

[26] D. Devane, C. Hamel, G. Gartlehner, B. Nussbaumer-Streit, U. Griebler, L. Affengruber, K. Saif-Ur-Rahman, and C. Garritty, "Key concepts in rapid reviews: an overview," *Journal of Clinical Epidemiology*, vol. 175, 2024.

[27] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Berlin, Germany: Springer, 2 ed., 2024.

[28] T. Gorschek, P. Garre, S. Larsson, and C. Wohlin, "A model for technology transfer in practice," *IEEE Software*, vol. 23, no. 6, pp. 88–95, 2006.

[29] C. Seaman, "Qualitative methods in empirical studies of software engineering," *IEEE Transactions on Software Engineering*, vol. 25, no. 4, pp. 557–572, 1999.

[30] M. T. Samarta, A. A. Santoso Gunawan, and M. E. Syahputra, "Systematic literature review and comparative performance analysis of sql and nosql databases in big data applications," pp. 218 – 222, 2024.

[31] J. Klein, I. Gorton, N. A. Ernst, P. Donohoe, K. Pham, and C. Matser, "Performance evaluation of nosql databases: A case study," *PABS 2015 - Proceedings of the 1st ACM/SPEC International Workshop on Performance Analysis of Big Data Systems*, pp. 5 – 10, 2015.

[32] F. Leal and M. A. Musicante, "Using sla to guide database transition to nosql on the cloud: A systematic mapping study," vol. 2016-July, 2016.

[33] C. Hadjichristofi, S. Diochnos, K. P. Andresakis, and V. C. Vescoukis, "Using time-series databases for energy data infrastructures †," *Energies*, vol. 17, no. 21, 2024.

[34] J. A. Béjar-Martos, A. J. Rueda-Ruiz, C. J. Ogayar-Anguita, R. J. Segura-Sánchez, and A. López, "Strategies for the storage of large lidar datasets—a performance comparison," *Remote Sensing*, vol. 14, no. 11, 2022.

[35] S. B. Kenitar, M. Arioua, and M. Yahyaoui, "A novel approach of latency and energy efficiency analysis of iiot with sql and nosql databases communication," *IEEE Access*, vol. 11, pp. 129247 – 129257, 2023.

[36] M. Hemmatpour, B. Montrucchio, M. Rebaudengo, and M. Sadoghi, "Analyzing in-memory nosql landscape," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 4, pp. 1628 – 1643, 2022.

[37] F. Souza, E. Tavares, and C. Araújo, "A modelling approach for estimating energy consumption of nosql-based storage systems," *Journal of Supercomputing*, vol. 81, no. 6, 2025.

[38] Z. Zhang, A. Megargel, and L. Jiang, "Performance evaluation of newsql databases in a distributed architecture," *IEEE Access*, vol. 13, pp. 11185 – 11194, 2025.

[39] D. Almeida, M. Lopes, L. Saraiva, M. Abbasi, P. G. Martins, J. L. Silva, and P. Váz, "Performance comparison of redis, memcached, mysql, and postgresql: A study on key-value and relational databases," pp. 902 – 907, 2023.

[40] D. Yedilkhan, A. K. Mukasheva, D. Bissengaliyeva, and Y. Suynullayev, "Performance analysis of scaling nosql vs sql: A comparative study of mongodb, cassandra, and postgresql," pp. 479 – 483, 2023.

[41] M. Salahat, M. A. Raza, Z. Khawar, J. A. Malik, H. Raza, H. K. Nair, and M. I. M. Jarrah, "Analysis of query processing on different databases," 2023.

[42] T. T. T. Do, T. B. Mai-Hoang, V. Q. Nguyen, and Q. T. Huynh, "Query-based performance comparison of graph database and relational database," *ACM International Conference Proceeding Series*, pp. 375 – 381, 2022.

[43] P. Kotiranta, M. Junkkari, and J. Nummenmaa, "Performance of graph and relational databases in complex queries," *Applied Sciences (Switzerland)*, vol. 12, no. 13, 2022.

[44] C. A. Györödi, D. V. Dumşe-Burescu, R. D. Zmaranda, and R. c. Győrödi, "A comparative study of mongodb and document-based mysql for big data application data management," *Big Data and Cognitive Computing*, vol. 6, no. 2, 2022.

[45] T. Kaneva and I. M. Valova, "Performance study of mysql and mongodb for IoT data processing and storage," pp. 60 – 63, 2022.

[46] E. Guney and N. Ceylan, "Response times comparison of mongodb and postgresql databases in specific test scenarios," *Lecture Notes of the Institute for Computer Sciences, Social-Informatics and Telecommunications Engineering, LNICST*, vol. 436, pp. 178 – 188, 2022.

[47] K. Mavrogiorgos, A. Kiourtis, A. Mavrogiorgou, and D. P. Kyriazis, "A comparative study of mongodb, arangodb and couchdb for big data storage," *ACM International Conference Proceeding Series*, pp. 8 – 14, 2021.

[48] A. Gomes, V. Lopes, E. D. O. Ribeiro, J. Lima, W. Costa, L. P. F. García, and M. T. Holanda, "An empirical performance comparison between mysql and mongodb on analytical queries in the comex database," 2021.

[49] C. A. Györödi, D. V. Dumşe-Burescu, R. D. Zmaranda, R. c. Győrödi, G. A. Gabor, and G. D. Pecherle, "Performance analysis of nosql and relational databases with couchdb and mysql for application's data storage," *Applied Sciences (Switzerland)*, vol. 10, no. 23, pp. 1 – 21, 2020.

[50] I. Ballas, V. D. Tsakanikas, E. Pefanis, and V. T. Tampakas, "Assessing the computational limits of graphdbs' engines - a comparison study between neo4j and apache spark," *ACM International Conference Proceeding Series*, pp. 428 – 433, 2020.

[51] P. Filip and L. Čegan, "Comparison of mysql and mongodb with focus on performance," pp. 184 – 187, 2020.

[52] M. M. Eyada, W. E. Saber, M. M. El Genidy, and F. A. E. Amer, "Performance evaluation of iot data management using mongodb versus mysql databases in different cloud environments," *IEEE Access*, vol. 8, pp. 110656 – 110668, 2020.

[53] E. S. Pramukantoro, D. P. Kartikasari, and R. A. Siregar, "Performance evaluation of mongodb, cassandra, and hbase for heterogenous iot data storage," pp. 203 – 206, 2019.

[54] B. Sirish Shetty and K. C. Akshay, "Performance analysis of queries in rdbms vs nosql," pp. 1283 – 1286, 2019.

[55] M. Ohyver, J. V. Moniaga, I. Sungkawa, B. E. Subagyo, and I. A. Chandra, "The comparison firebase realtime database and mysql database performance using wilcoxon signed-rank test," vol. 157, pp. 396 – 405, 2019.

[56] R. Deari, X. Zenuni, J. Ajdari, F. Ismaili, and B. Raufi, "Analysis and comparision of document-based databases with relational databases: Mongodb vs mysql," *2018 International Conference on Information Technologies, InfoTech 2018 - Proceedings*, 2018.

[57] D. A. Pereira, W. Ourique de Morais, and E. P. Pignaton De Freitas, "Nosql real-time database performance comparison," *International Journal of Parallel, Emergent and Distributed Systems*, vol. 33, no. 2, pp. 144 – 156, 2018.

[58] C. Messaoudi, R. Fissoune, and H. Badir, "A performance evaluation of nosql databases to manage proteomics data," *International Journal of Data Mining and Bioinformatics*, vol. 21, no. 1, pp. 70 – 89, 2018.

[59] K. Kaur and M. Sachdeva, "Performance evaluation of newsql databases," 2017.

[60] R. Poljak, P. Poščić, and D. Jaksic, "Comparative analysis of the selected relational database management systems," pp. 1496 – 1500, 2017.

[61] S. Gupta and R. Rani, "A comparative study of elasticsearch and couch db document oriented databases," vol. 1, 2016.

[62] M. Kendea, V. A. Gkantouna, A. Rapti, S. A. Sioutas, G. E. Tzimas, and D. Tsolis, "Graph dbs vs. column-oriented stores: A pure performance comparison," *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 9511, pp. 62 – 74, 2016.

[63] M. R. M. Chopade and N. S. Dhavase, "Mongodb, couchbase: Performance comparison for image dataset," in *2017 2nd International Conference for Convergence in Technology (I2CT)*, pp. 255–258, April 2017.

[64] A. Mahgoub, S. Ganesh, F. Meyer, A. Grama, and S. Chaterji, "Suitability of nosql systems — cassandra and scylladb — for iot workloads," in *2017 9th International Conference on Communication Systems and Networks (COMSNETS)*, pp. 476–479, 2017.

Appendix A

Survey responses

Table A.1: Expert survey responses.

Question	Response
Q1	Strengths: Clear paths, seemingly simple decisions. Weaknesses: Flow diagrams like these do not really cater for "weighing the options" (aka. conflicting forces). They assume that decision points come in well-defined sequences. Alternatives could be point-systems, possibly with ranges, or purely qualitative forms (though that would be very open to subjective decisions, as always with qualitative analyses)
	It is good start, but I think it It oversimplifies DBMS selection by reducing it to a few decisions. I think realworld DBMS selection involves many more dimensions like: licensing cost, infrastructure, team expertise, data volume, consistency requirements, compliance/regulatory needs, and whether it can handle both vector and non-vector workloads ... .The framework also completely omits major enterprise DBMS platforms most notably Oracle Database, which is the world's leading RDBMS and now supports native vector operations (Oracle 23ai with AI Vector Search that improved alot in 26ai). A database like Oracle eliminates the need to select a separate VDBMS entirely. Pros: clean design and clearly explains the flow for vector and non vector. Cons: really missing the leader, Oracle the top one, Postgress ... Oracle with native vector search, predictive queries, RAC, for horizontal and GEO scaling ...

Table A.1: Expert survey responses. *(continued)*

Question	Response
	Its on a too high-level, selecting databases for any system/product must consider a lot more variables and features than what is presented here. Also its quite unclear/undefined why vector-data is so heavily featured here. Another huge aspect that is missing from this is the kinds of query patterns and datamodelling approaches that must be supported, this depends greatly on the kinds of data/usecases you have when selecting the database. Another omission is the idea of what consistency guarantees are required, ie its not just a trade-off on latency/throughput but also what kind of consistency-mechanisms/semantics I can get from the database (for example Cassandra has tunable consistency per-query). The flowchart also ignores that selecting a database (or any 3PP for that matter) is also a question of sourcing (buy vs build), granted you don't often build a database but there are many different 3PP databases and you must also compare the project stability/support/LCM/community aspects when selecting. A database could have the best tech in the whole world but if no-one is using it and there is only one person maintaining it you could never select that as an acceptable alternative (extreme example but just to illustrate the point, its a spectrum).
Q2	It might very well be useful, but it is of course limited to the options stated in the framework.
	It is good, but really limited. It can help only to select between those mentioned DBs, while in huge OLTP env, top ones should be included like Oracle.
	No
Q3	Yes, I would say so
	yes it is clear and easy to follow.
	No, also i dont understand what you mean by "is it clear that a vector/non-vector combination of databases is possible"
Q4	Probably, but I am still stuck in an old, classical, enterpricey RDBMS... But those are also developing their scaling capabilities and vector features...
	Oracle database 26ai Postgress
	Yes, see my answer on the first question

Table A.1: Expert survey responses. *(continued)*

Question	Response
Q5	Interesting to see the decisions being presented as a flow diagram. So a nice touch for the method!
	As it mentioned earlier, the flow and design is well, but it is really limited and missing the top ones.
	I am not sure how to effectively use the framework to gain any insights into this. But selecting databases for a complex system is also one of the most difficult tasks in architecture and it sets you up for potential failures you had no way of considering, I don't envy the task of creating a simple one-diagram framework for accomplishing what basically takes years of experience with such systems and familiarity with the underlying technology to do well.



Faculty of Software Engineering, Blekinge Institute of Technology, 371 79 Karlskrona, Sweden